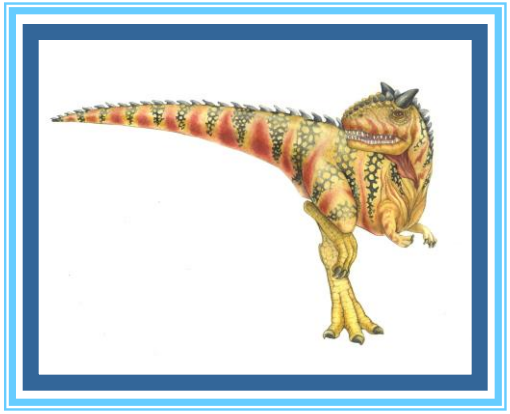
Chapter 2: Operating-System Services



Silberschatz, Galvin and Gagne ©2018



Outline

- Operating System Services
- User and Operating System-Interface
- System Calls
- System Services
- Linkers and Loaders
- Why Applications are Operating System Specific
- Design and Implementation
- Operating System Structure
- Building and Booting an Operating System
- Operating System Debugging





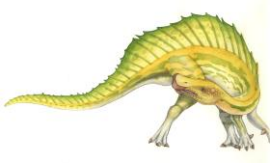
Objectives

- Identify services provided by an operating system
- Illustrate how system calls are used to provide operating system services
- Compare and contrast monolithic, layered, microkernel, modular, and hybrid strategies for designing operating systems
- Illustrate the process for booting an operating system
- Apply tools for monitoring operating system performance
- Design and implement kernel modules for interacting with a Linux kernel

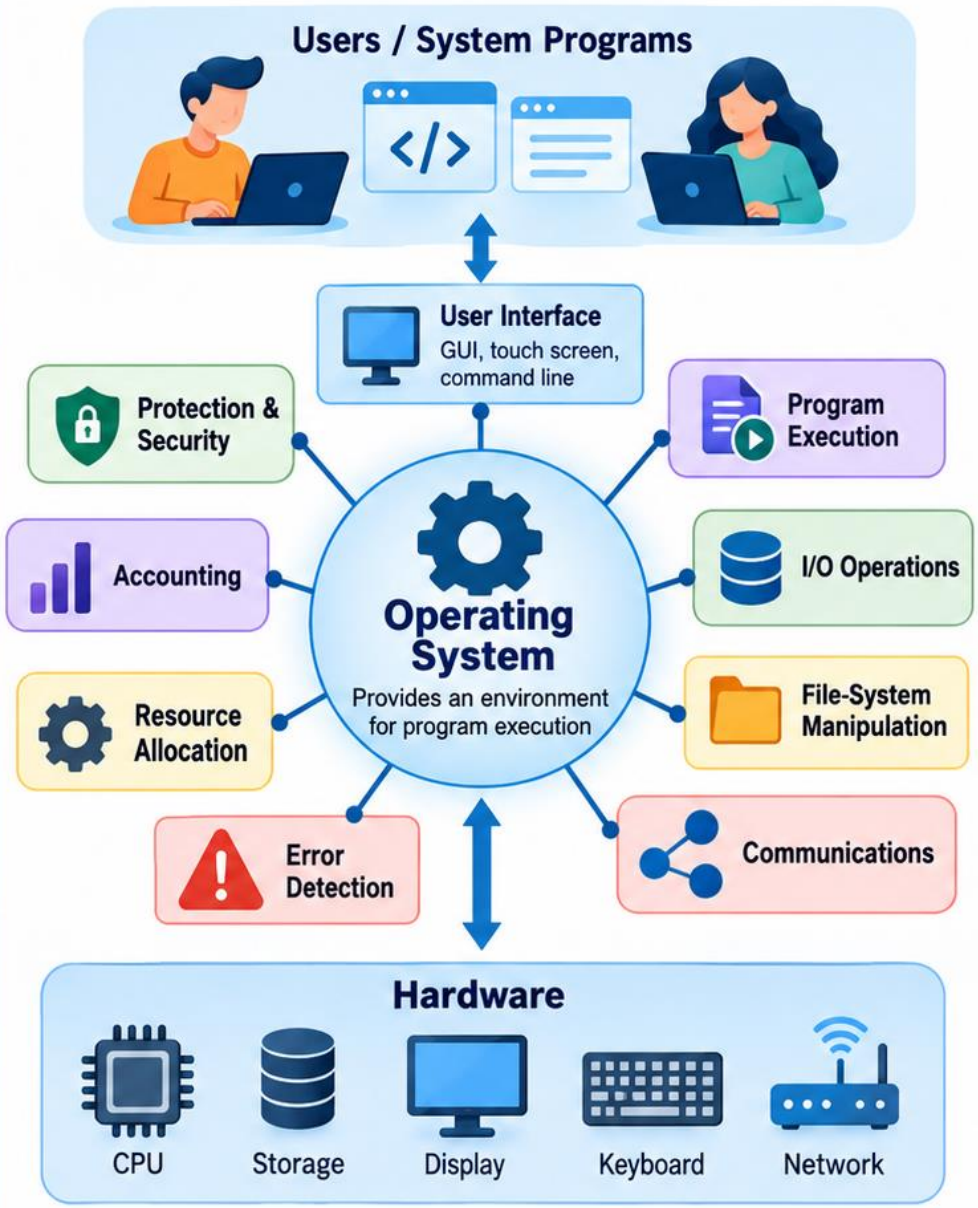




Operating System Services



Operating-System Services



What the OS Provides

- An environment for the execution of programs
- Services for users and application programs
- Makes system use and programming easier

User-Oriented Services

- User interface (GUI, touch screen, command line)
- Program execution
- I/O operations
- File-system manipulation
- Communications
- Error detection

System-Oriented Services

- Resource allocation
- Accounting
- Protection and security

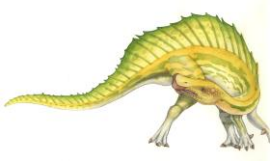
Key Idea

- Specific services differ from one operating system to another
- These common service classes appear in most systems
- They help organize how programs interact with hardware



Operating System Services

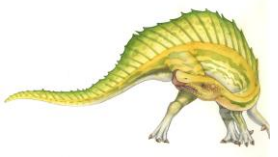
- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (**UI**).
    - Varies between **Command-Line** (**CLI**), **Graphics User Interface** (**GUI**), **touch-screen**, **Batch**
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
  - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.





Operating System Services (Cont.)

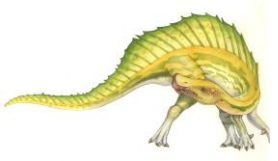
- One set of operating-system services provides functions that are helpful to the user (Cont.):
  - **Communications** – Processes may exchange information, on the same computer or between computers over a network
    - Communications may be via shared memory or through message passing (packets moved by the OS)
  - **Error detection** – OS needs to be constantly aware of possible errors
    - May occur in the CPU and memory hardware, in I/O devices, in user program
    - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
    - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

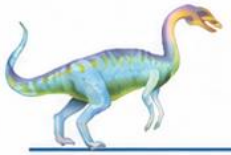




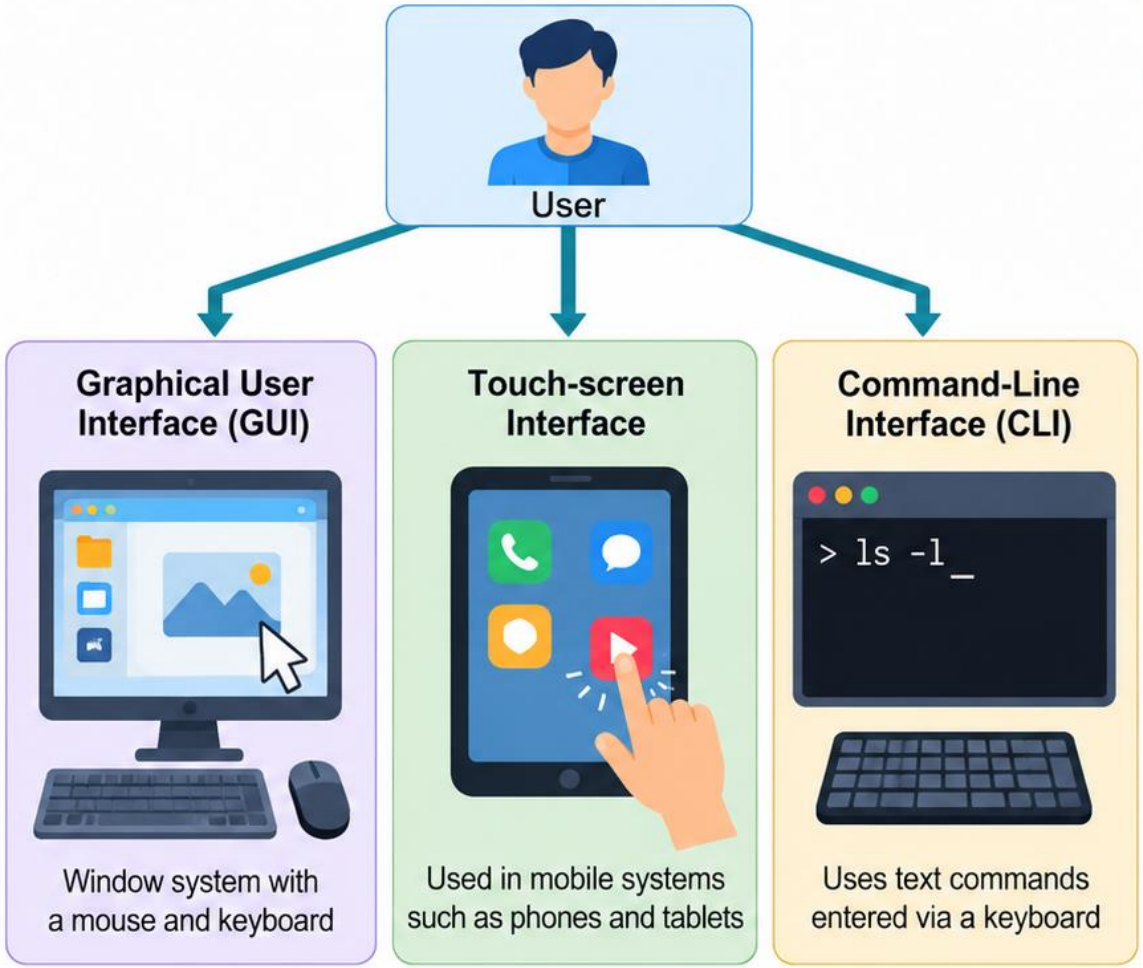
Operating System Services (Cont.)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
  - **Logging -** To keep track of which users use how much and what kinds of computer resources
  - **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
    - **Protection** involves ensuring that all access to system resources is controlled
    - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts



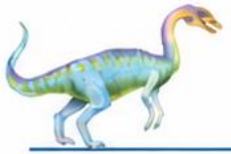


User Interface

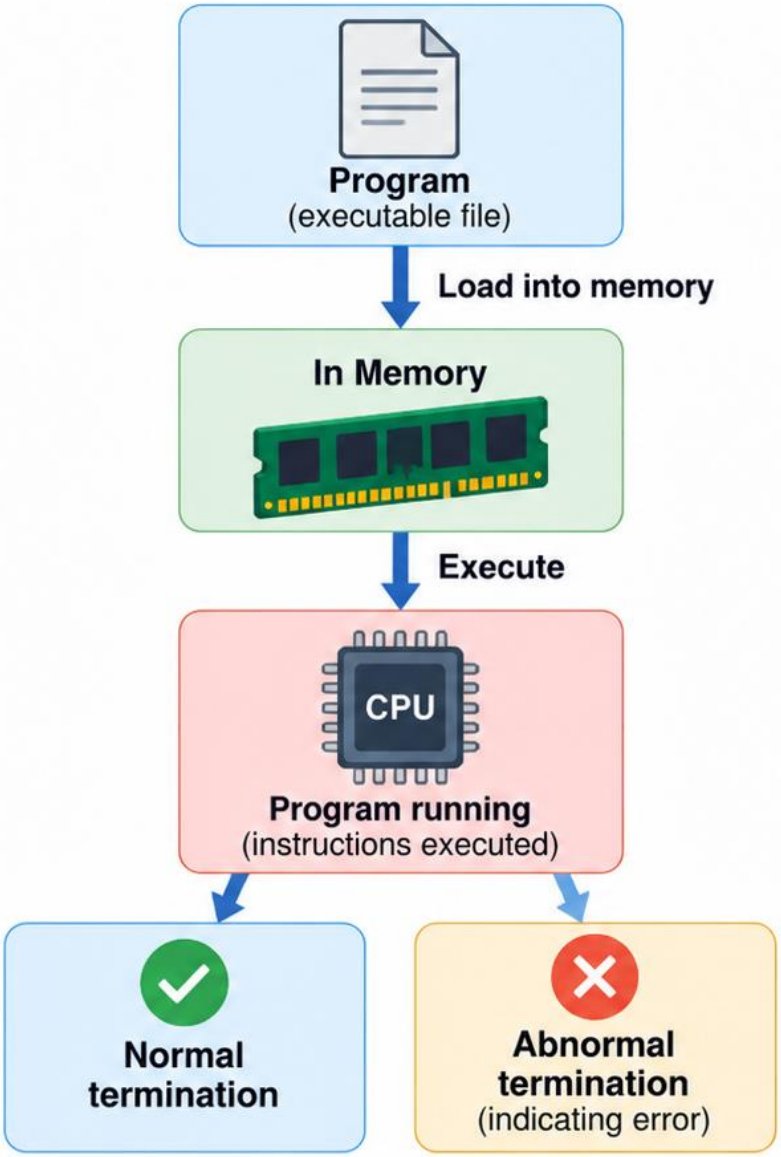


- A **user interface (UI)** allows users to interact with the operating system.
- The most common form is a **graphical user interface (GUI)**.
  - A window system with a mouse to direct I/O, choose from menus, and make selections.
  - A keyboard to enter text.
- Mobile systems such as phones and tablets provide a **touch-screen interface**.
  - Users slide their fingers across the screen or press on-screen buttons to select choices.
- Another option is a **command-line interface (CLI)**.
  - Uses text commands.
  - A keyboard is used to type commands in a specific format with specific options.
- Some systems provide two or all three of these variations.





Program Execution

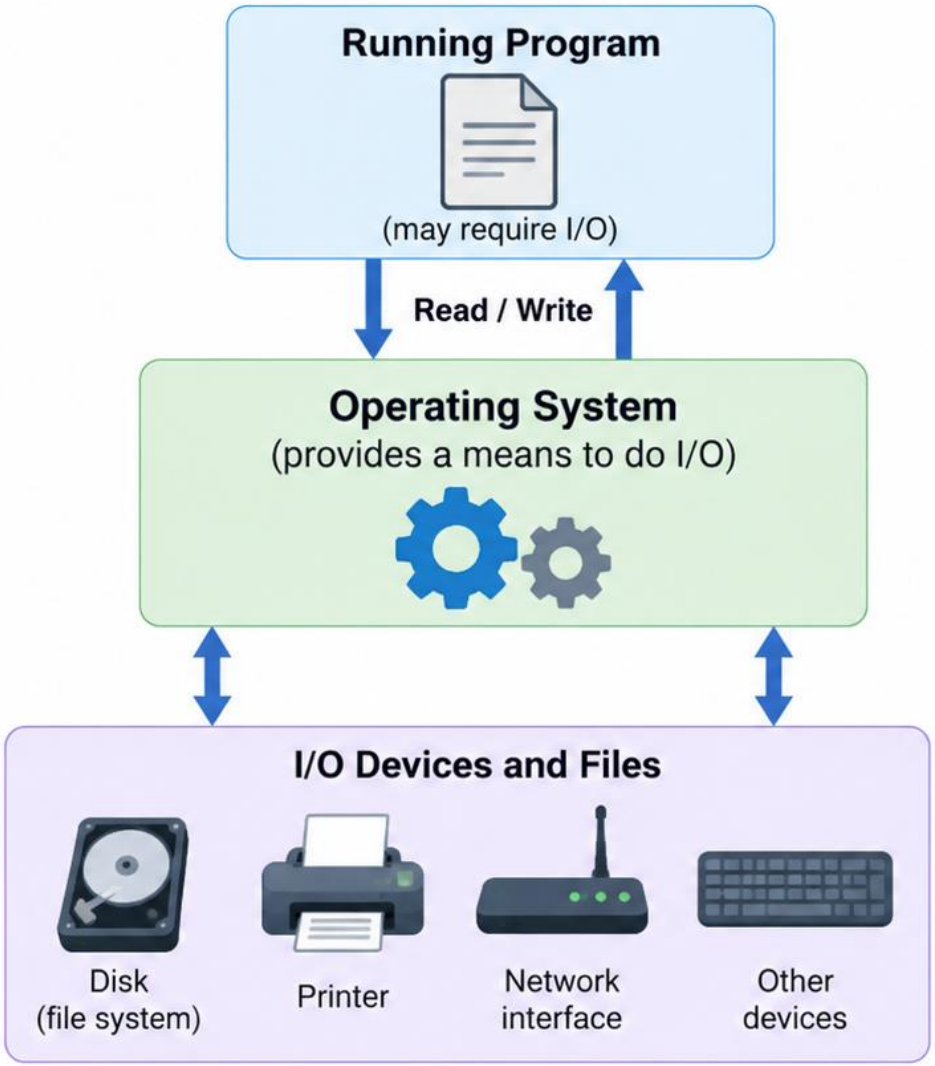


- **Program execution** – The operating system must be able to load a program into memory and to run that program.
- The system **loads the program** from secondary storage into main memory, allocates necessary resources, and starts its execution.
- While executing, the program uses the **CPU**, **memory**, and other **system resources** under the control of the operating system.
- The program must be able to end its execution, either **normally** or **abnormally** (indicating an error).





I/O Operations

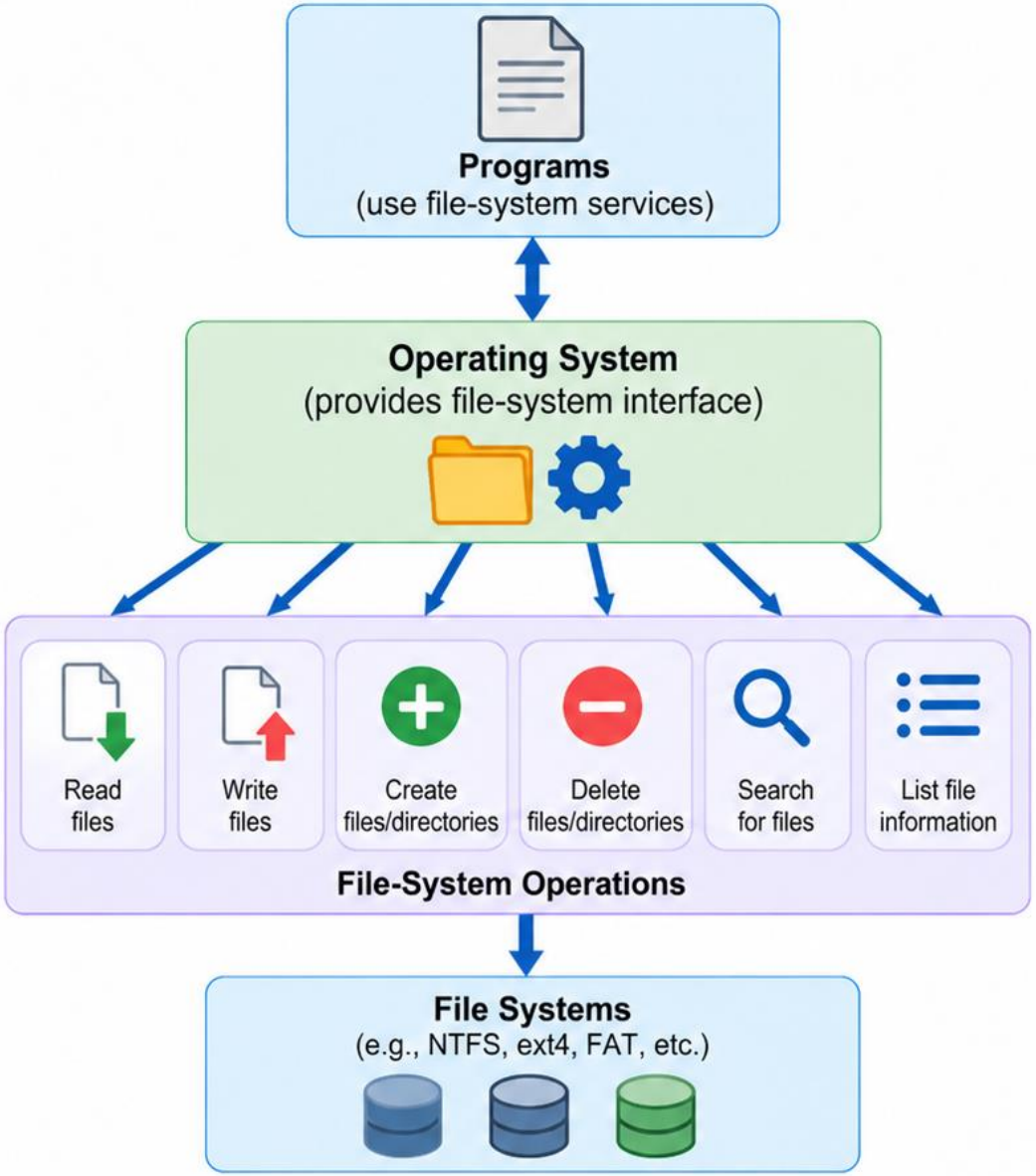


- A running program may require **I/O**, which may involve a file or an I/O device.
- For specific devices, **special functions** may be desired (such as reading from a network interface or writing to a file system).
- For efficiency and protection, users usually **cannot control I/O devices directly**.
- Therefore, the **operating system** must **provide a means to do I/O**.





File-System Manipulation

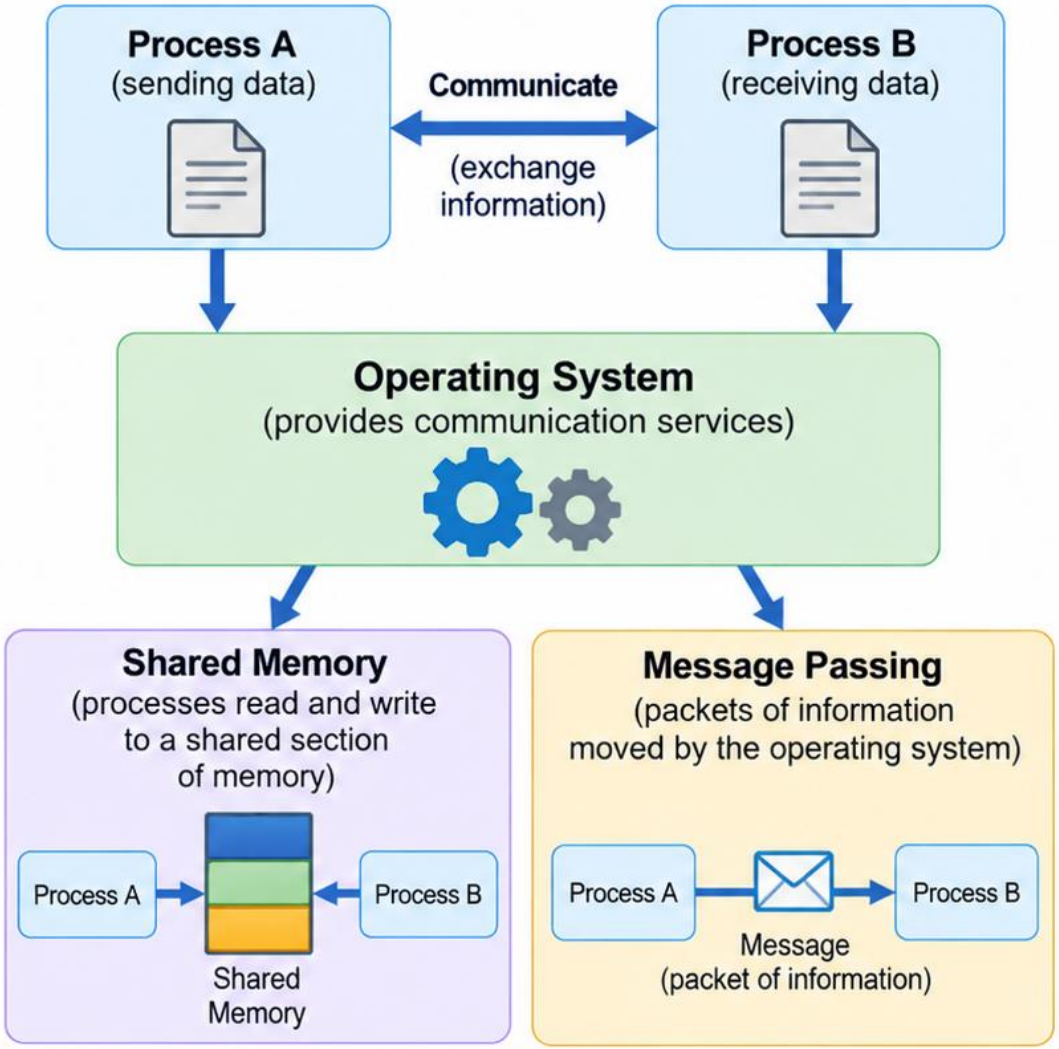


- The **file system** is of particular interest.
- Programs need to **read and write files and directories**.
- They also need to **create and delete** them by name, **search** for a given file, and **list file information**.
- Some operating systems include **permissions management** to allow or deny access to files or directories based on file ownership.
- Many operating systems provide a **variety of file systems**, sometimes to allow personal choice and sometimes to provide specific features or performance characteristics.





Communications



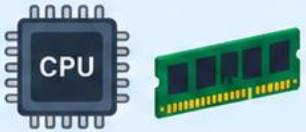
- There are many circumstances in which one process needs to **exchange information** with another process.
- Communication may occur between processes that are executing on the **same computer** or between processes that are executing on **different computer systems** tied together by a network.
- Communications may be implemented via **shared memory**, in which two or more processes read and write to a shared section of memory.
- Or via **message passing**, in which packets of information in predefined formats are moved between processes by the operating system.





Error Detection

CPU and Memory Hardware Errors



- Memory error
- Power failure

I/O Device Errors



- Parity error on disk
- Connection failure on a network
- Lack of paper in the printer

User Program Errors



- Arithmetic overflow
- Attempt to access an illegal memory location

Operating System
(detects and handles errors)





Correct the error
(when possible)



Halt the system
(if necessary)



Terminate the error-causing process



Return an error code
(for the process to handle)

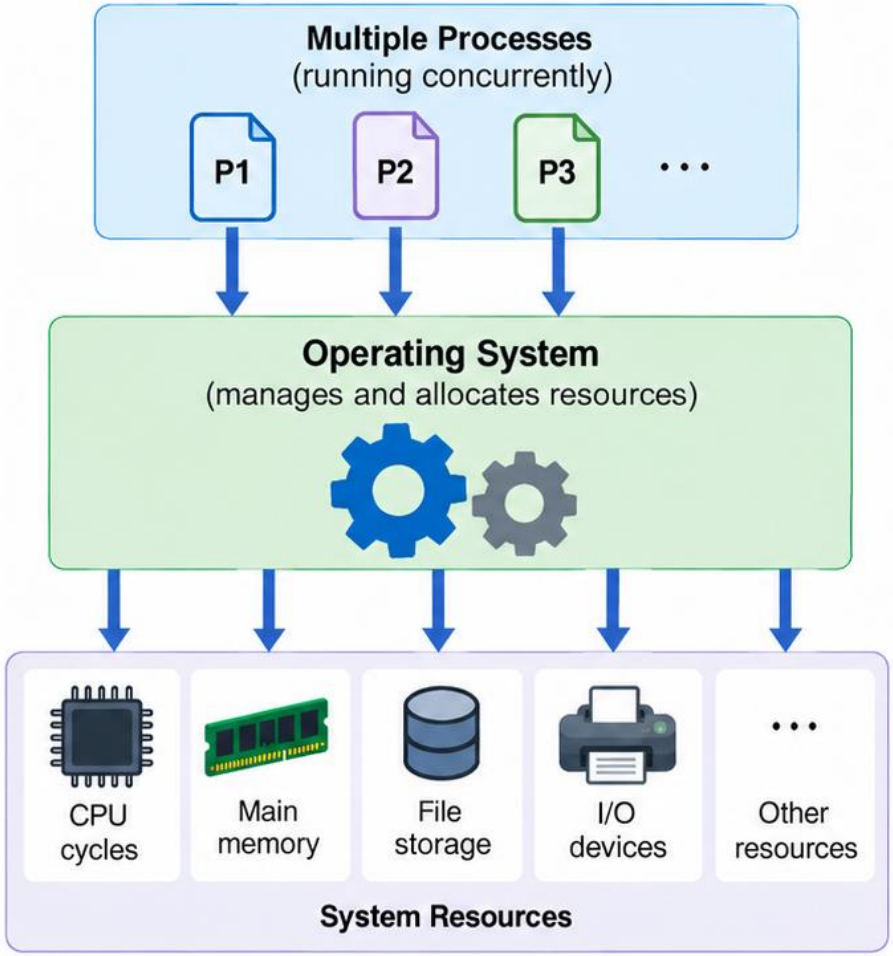
- The operating system needs to be **detecting and correcting errors constantly**.
- Errors may occur in the **CPU and memory hardware** (such as a memory error or a power failure), in **I/O devices** (such as a parity error on disk, a connection failure on a network, or lack of paper in the printer), and in the **user program** (such as an arithmetic overflow or an attempt to access an illegal memory location).
- For each type of error, the operating system should take the **appropriate action** to ensure correct and consistent computing.
- Sometimes, it has no choice but to **halt the system**. At other times, it might **terminate an error-causing process** or **return an error code** to a process for the process to detect and possibly correct.





System Efficiency Functions

Another set of operating-system functions exists not for helping the user but rather for ensuring the **efficient operation of the system itself**. Systems with multiple processes can gain efficiency by sharing the computer resources among the different processes.



- **Resource allocation** – When multiple processes are running, resources must be allocated to each of them. The operating system manages many types of resources, such as CPU cycles, main memory, file storage, and I/O devices. It uses specialized allocation code (e.g., **CPU-scheduling routines**) to determine how best to use resources, considering factors like CPU speed, number of cores, and the processes to be executed.

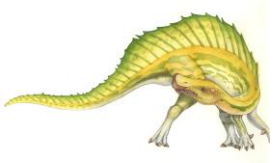
- **Logging** – The operating system keeps track of which programs use how much and what kinds of resources. This record keeping may be used for **accounting** (e.g., to bill users) or for accumulating **usage statistics**, which help system administrators reconfigure the system to improve services.

- **Protection and security** – The operating system controls access to information and system resources. It prevents processes from interfering with each other or with the operating system, enforces **access control**, and requires **user authentication** (e.g., passwords). Security also defends against external threats (e.g., invalid network access) and records connections to detect break-ins. Precautions must be applied throughout the system, as a chain is only as strong as its weakest link.





User and Operating-System Interface





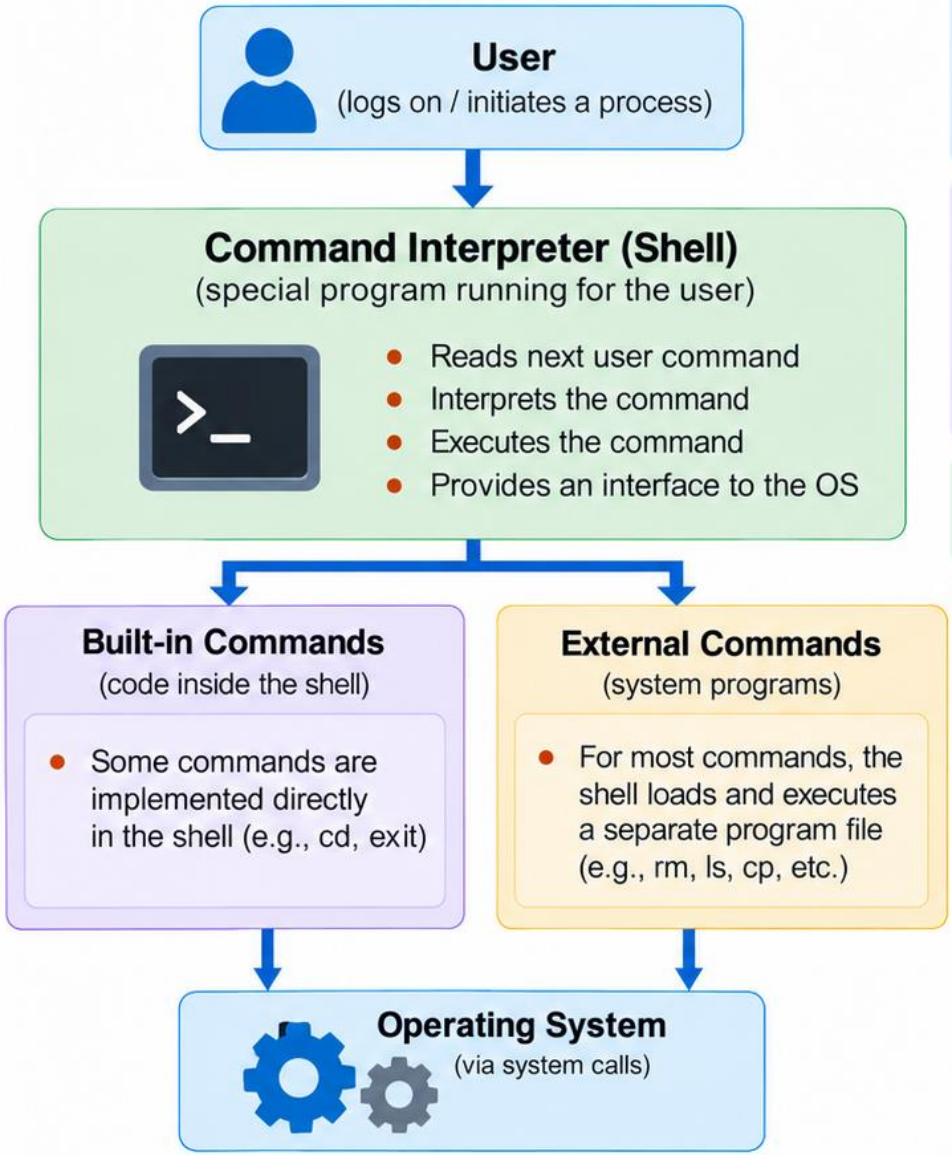
Command Line interpreter

- CLI allows direct command entry
- Sometimes implemented in kernel, sometimes by systems program
- Sometimes multiple flavors implemented – **shells**
- Primarily fetches a command from user and executes it
- Sometimes commands built-in, sometimes just names of programs
  - If the latter, adding new features doesn't require shell modification





Command Interpreter (Shell)



- Most operating systems, including Linux, UNIX, and Windows, treat the **command interpreter** as a special program that is running when a process is initiated or when a user first logs on.
- On systems with multiple command interpreters, the interpreters are known as **shells**. On UNIX and Linux, a user may choose among several shells, including the Cshell, **Bourne-Again shell** (**bash**), Korn shell, and others. Third-party and user-written shells are also available.
- Most shells provide similar functionality, and a user's choice is generally based on personal preference.
- The main function of the command interpreter is to get and execute the next user-specified command, many of which manipulate files (create, delete, list, print, copy, execute, etc.).

```
bash — -zsh — 80×24
Last login: Mon Apr 22 10:24:17 on ttys000
user@macbook ~ % ls
Desktop        Documents      Downloads      Pictures
user@macbook ~ % pwd
/Users/user
user@macbook ~ % rm file.txt
user@macbook ~ %
```

Example: bash shell on macOS

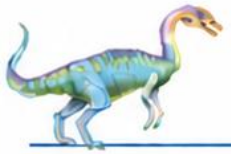




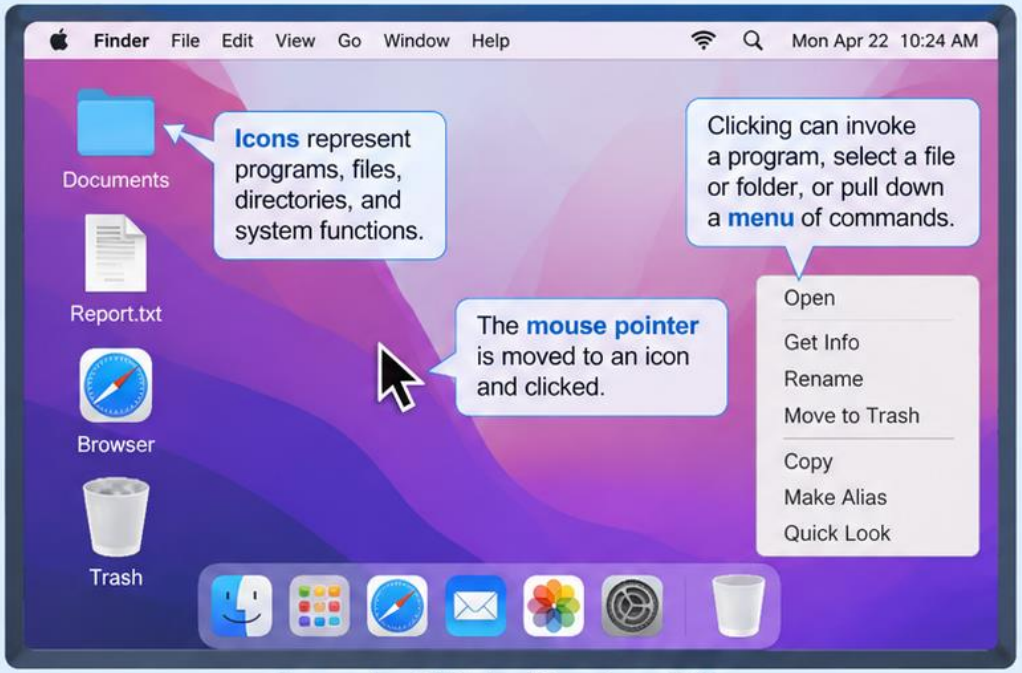
User Operating System Interface - GUI

- User-friendly **desktop** metaphor interface
  - Usually mouse, keyboard, and monitor
  - **Icons** represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**)
  - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
  - Microsoft Windows is GUI with CLI "command" shell
  - Apple Mac OS X is "Aqua" GUI interface with UNIX kernel underneath and shells available
  - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)





Graphical User Interface (GUI)



Example: GUI desktop (macOS)

Evolution of GUI Interfaces



1973
Xerox Alto
First GUI
(Xerox PARC)

1980s
Apple Macintosh
Made GUIs
widespread

1985
Windows 1.0
GUI added
to MS-DOS

Today
KDE / GNOME
Open-source GUIs
on Linux/UNIX

- A **graphical user interface (GUI)** is a user-friendly way to interact with the operating system, using a mouse-based window and-menu system characterized by a **desktop metaphor**.
- The user moves the mouse to position its pointer on images, or **icons**, on the screen that represent programs, files, directories, and system functions.
- Depending on the mouse pointer's location, clicking a button can invoke a **program**, select a file or directory (**folder**), or pull down a **menu** that contains commands.
- GUI interfaces first appeared from research at **Xerox PARC** in the early 1970s, with the first GUI on the **Xerox Alto** in 1973, and became widespread with the **Apple Macintosh** in the 1980s.
- Microsoft's first version of **Windows** (1.0) was a GUI added to MS-DOS. Later versions introduced major design changes and functional enhancements.
- Although UNIX systems were traditionally command-line based, GUI interfaces like **KDE** and **GNOME** are available and widely used on Linux and UNIX systems under open-source licenses.



Touch-Screen Interface

Touch-screen interfaces are common in smartphones and **tablets.**
Users interact with the system through **finger gestures** on the screen.
Most devices also provide an **on-screen keyboard**.



Example: smartphone touch-screen interface

Common Touch Gestures



Tap
Press briefly to select an item.



Swipe
Drag your finger across the screen.



Scroll
Swipe up or down to scroll content.



Pinch
Pinch in or out to zoom.



Long press
Touch and hold for more options.

Why used?

- Practical for mobile devices
- No mouse-and-keyboard required

How users interact

- Press, swipe, and drag on the screen
- Select apps and options directly

Example

- iPhone and iPad use the Springboard interface
- Modern phones simulate a keyboard on screen



Choice of Interface

Choice is mostly personal preference

- System administrators and power users often prefer the **command-line interface (CLI)** because it is faster and more efficient.
- Some systems provide only a subset of functions in the GUI, leaving advanced tasks to command-line users.

Command-Line Interface (CLI)



- Faster access
- Easier for repetitive tasks
- Supports shell scripts

Graphical User Interface (GUI)



- User-friendly
- Popular with most users
- Available on Windows, macOS (Aqua), etc.

Touch-Screen Interface



- Used in smartphones and tablets
- Interact by touch gestures
- Rarely use command-line interfaces



Figure 2.3 The iPhone touch screen.



Figure 2.4 The macOS GUI.

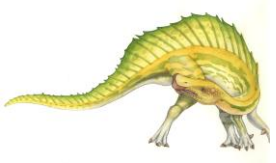
Key Points

- CLI is preferred by experts and supports **scripting**.
- Most Windows users rely on the GUI, while macOS provides both a GUI (Aqua) and a command-line interface.
- Mobile systems mainly use touch-screen interfaces.
- The user interface is separate from the core operating system, which provides services to user programs.

Silberschatz, Galvin and Gagne ©2018



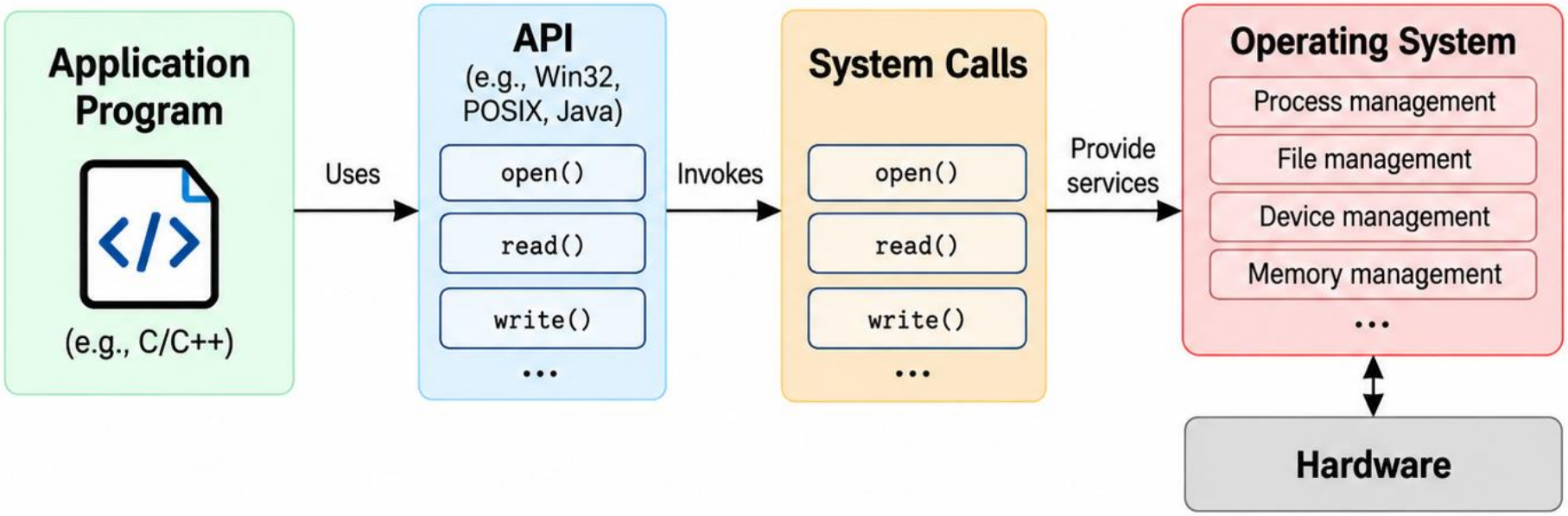
System Calls





System Calls

- **Programming interface** to the services provided by the OS
- Typically written in a **high-level language** (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use
- **Common APIs:** Win32 API (Windows), POSIX API (UNIX/Linux/macOS), and Java API (JVM)



Note: The system-call names used throughout this text are generic.



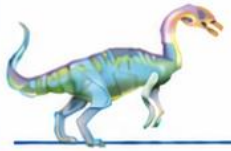


System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface** (**API**) rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

Note that the system-call names used throughout this text are generic





System Call Example: Copy a File

System calls provide an interface to the services made available by an operating system. They are usually available as functions in C and C++.

Example: UNIX cp command

```
cp in.txt out.txt
```

This command copies the input file `in.txt` to the output file `out.txt`.

Steps in Copying a File (using system calls)

1. **Get file names**
   (from command line, user input, or GUI)
2. **Open input file**
   (system call)
3. **Create and open output file**
   (system call)
4. **Read from input file**
   (system call)
5. **Write to output file**
   (system call)
6. **Handle errors**
   (e.g., file not found, permission denied)

1 Get file names

- File names can be given as command-line arguments (e.g., `cp in.txt out.txt`),
- entered by the user (keyboard input), or
- selected using a GUI (mouse and menus).

2 Open input file

- Use a system call to open the input file.
- If the file does not exist or is protected, output an error message and terminate the program.

3 Create and open output file

- Use a system call to create and open the output file.
- If a file with the same name already exists, options include:
  - Abort the program.
  - Delete the existing file and create a new one.
  - Ask the user (in an interactive system) whether to replace it.

4 Copy the data

- Read data from the input file (system call).
- Write the data to the output file (system call).

5 Handle errors

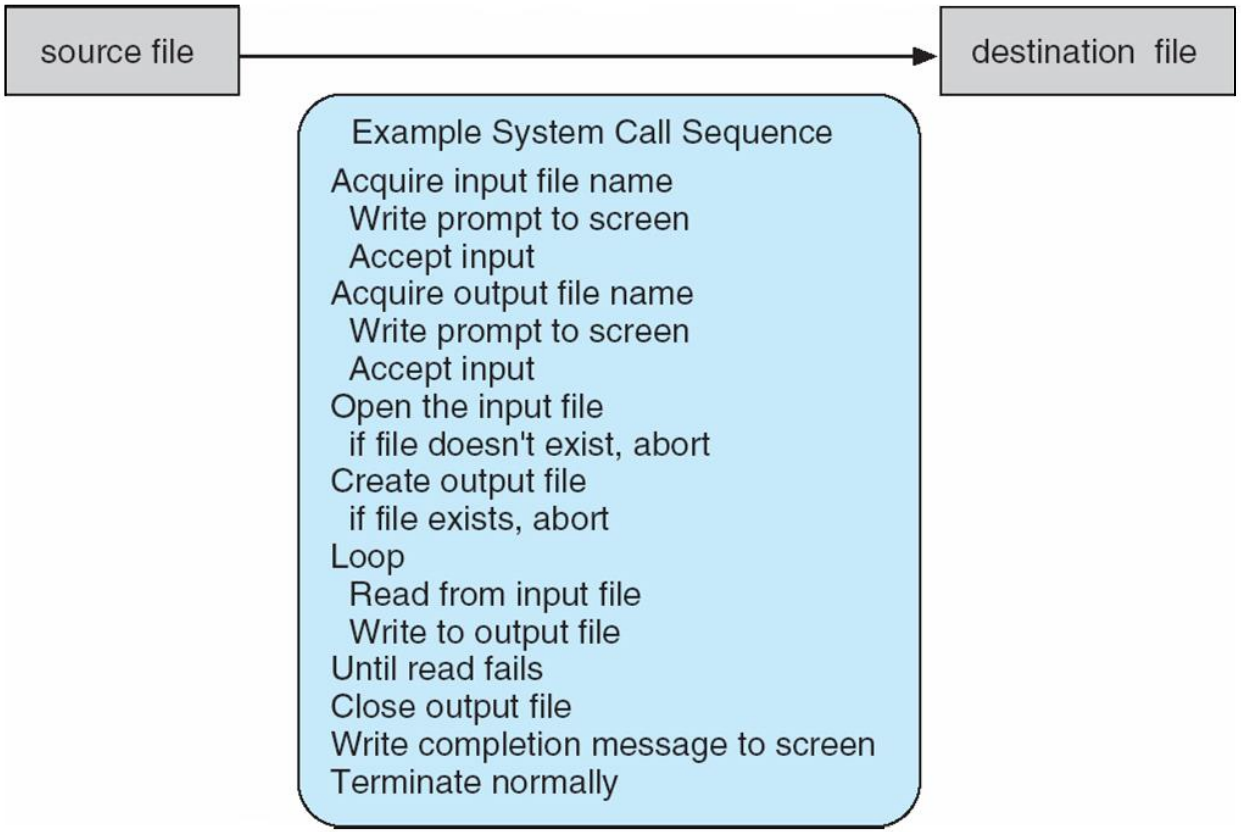
- Check for and handle possible errors at each step (e.g., file not found, permission denied, disk full) using appropriate system calls.

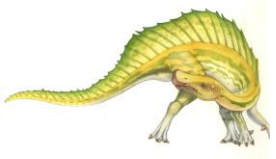




Example of System Calls

- System call sequence to copy the contents of one file to another file







What is an API?

An **API (Application Programming Interface)** is a set of rules, functions, methods, and protocols that allows one software program to communicate with and use the functionality of another software component without knowing its internal implementation.

1 Simple Example (C Library API)

Use the **sqrt()** function to calculate a square root.

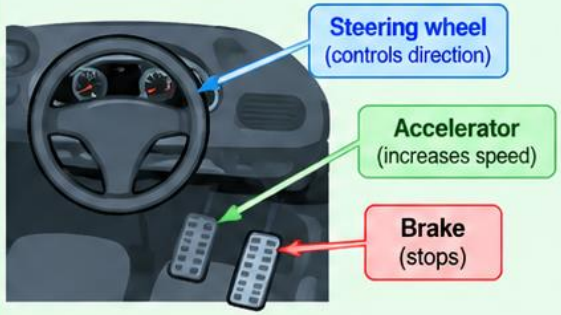
```c
#include <math.h>

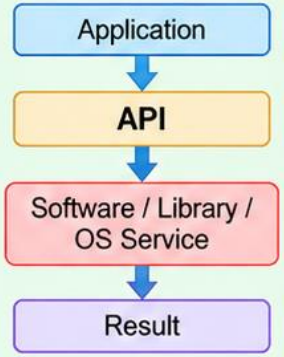
double result = sqrt(25);
```

sqrt() is part of the C Standard Library API. Your program does not need to know how the calculation is implemented.

2 API as an Interface (Car Analogy)

You use the steering wheel, accelerator, and brake without knowing how the engine works.





3A Operating System API

An OS provides APIs that applications use to request system services.

```c
// Windows API example
HANDLE hFile = CreateFile(...);
```

Your program uses the Windows API, while the operating system performs the actual operations internally.

Examples (Windows API)

- CreateFile()
- ReadFile()
- WriteFile()
- CreateProcess()

3B Library API

Libraries provide APIs containing reusable functions.

```c
// C Standard Library example
printf("Hello World");
int len = strlen("OS");
```

You can use these functions without knowing how they are implemented internally.

Examples (C Library)

- printf()
- scanf()
- malloc()
- strlen()
- sqrt()

Key Takeaway

An API provides a controlled interface through which a program can request services or data from another program, library, operating system, or service.





Application Programming Interface (API)

What is an API?

- An API defines a **set of functions** available to application programmers, including parameters and return values.
- Programmers use the API through a **library of code** provided by the operating system (e.g., `libc` in UNIX/Linux).

Common APIs



Windows API
for Windows systems



POSIX API
for UNIX, Linux, and macOS

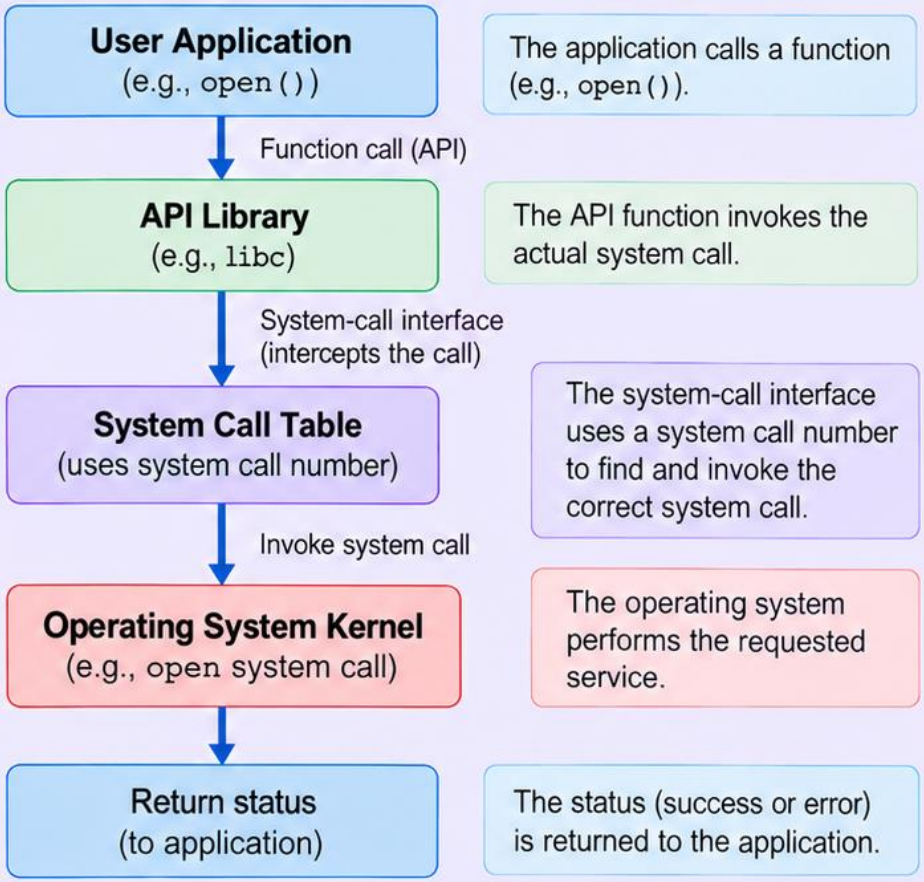


Java API
for programs on the Java virtual machine

Why use an API?

- Improves program portability.
- Easier to use than raw system calls.
- Often similar to the underlying system calls.

How an API Handles a System Call (open())



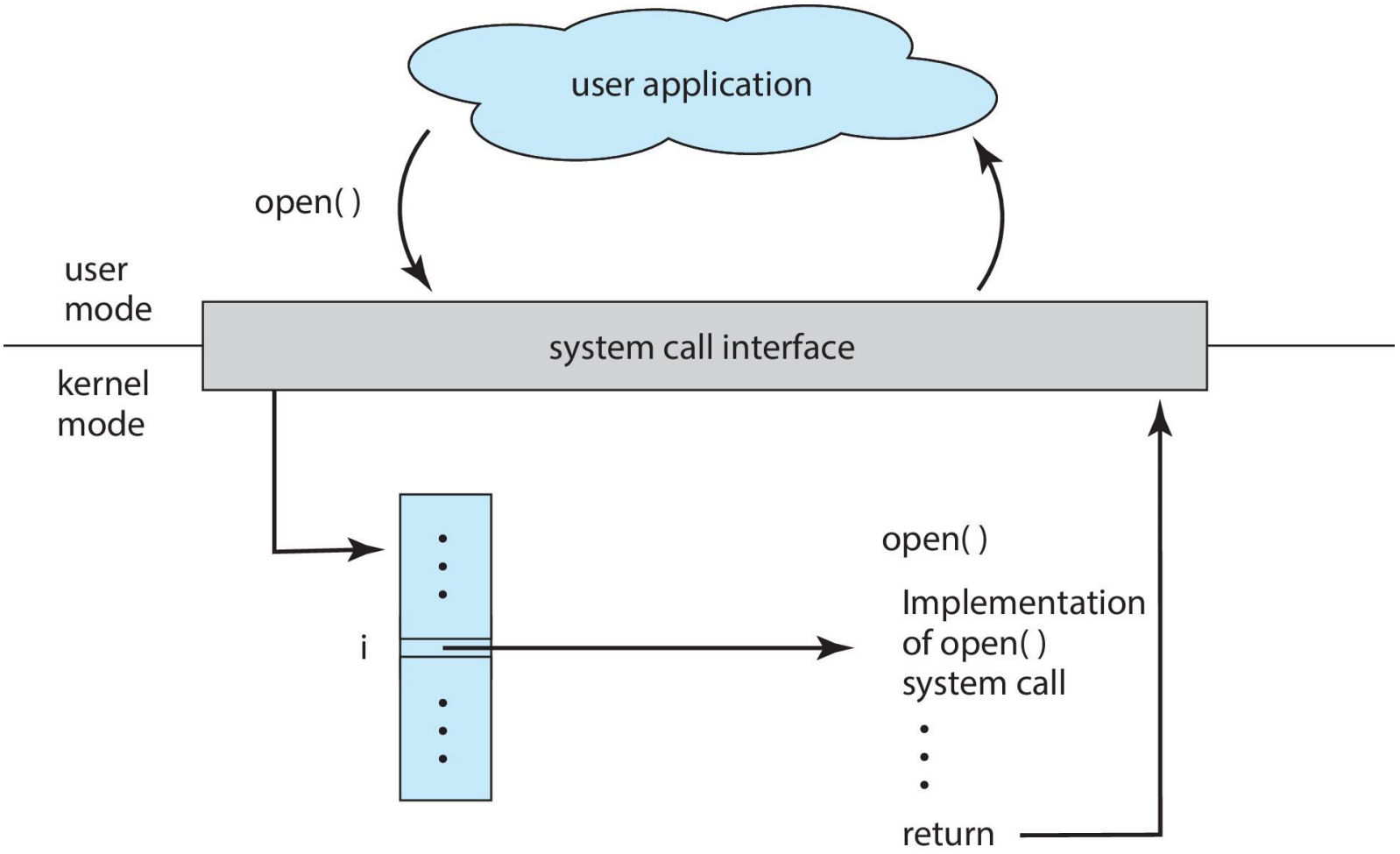
Key Takeaway

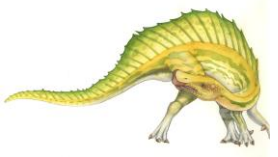
- The API and **run-time environment (RTE)** hide the low-level details of system calls, making it easier for programmers to develop applications.





API – System Call – OS Relationship





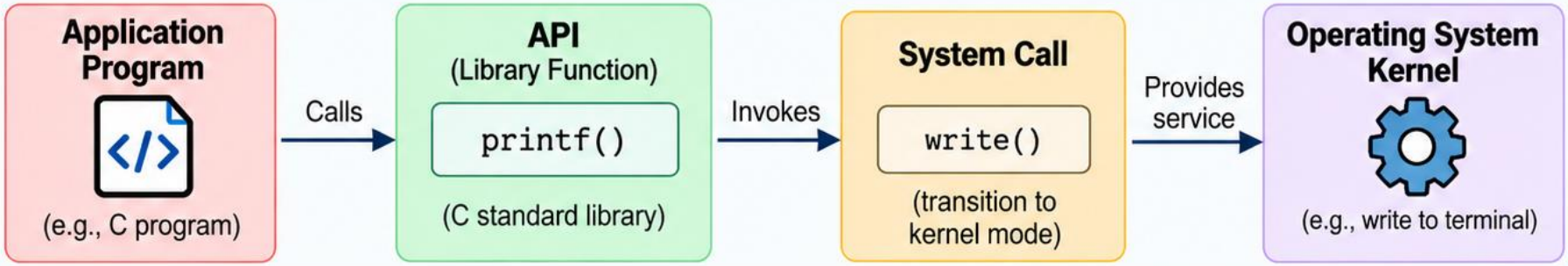


System Call vs API

	System Call	API (Application Programming Interface)
Definition	Mechanism through which a user program requests a service from the OS kernel.	Programming interface that provides functions/methods for application developers to use.
Mode of operation	Causes a transition from **user mode** to **kernel mode**.	May or **may not** involve a system call.
Provided by	Operating system / kernel.	Libraries, frameworks, or OS environments.
Examples	`fork()`, `read()`, `write()`, `open()` (POSIX/Linux).	C standard library functions (`printf()`, `malloc()`), Windows API functions, etc.
Level	Lower-level and closer to the OS.	Higher-level and easier for programmers to use.

Relationship

An API can internally invoke one or more system calls.





Key Takeaway

An **API** is the interface programmers use; a **system call** is the mechanism used to request a service directly from the OS kernel.



Example: Does an API Always Use a System Call?

1 Example 1: sqrt() (No system call)

Calculate the square root of a number in C.

```c
#include <math.h>
double result = sqrt(25);
```

Your Program
↓
sqrt()
(API / Library function)
↓
Math Library
(implements sqrt)
↓
CPU performs mathematical operations
↓
Result = 5

- **sqrt()** is part of the C Standard Library API.
- It is a computation performed by the CPU/ library.
- No need to enter the OS kernel.
- **No system call is involved.**

2 Example 2: open() (Uses a system call)

Open a file in C.

```c
#include <fcntl.h>
int fd = open("data.txt", O_RDONLY);
```

Your Program
↓
open()
(API / Library function)
↓
System Call
(transition to kernel mode)
↓
OS Kernel
(file system)
↓
File opened
(file descriptor returned)

- **open()** is provided as an API (or library interface).
- It invokes a system call to request the service from the OS kernel.
- The OS performs the file-system operation.
- **A system call is involved.**

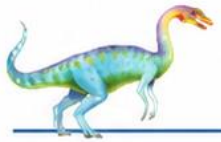
Comparison of Examples

Function / Example	API?	System call involved?	Purpose
`sqrt(25)`	Yes	No	Mathematical computation
`strlen("Hello")`	Yes	No	String manipulation
`printf("Hello")`	Yes	Usually yes (indirectly)	Output to terminal (uses `write()`)
`open("file.txt")`	Yes (interface)	Yes	Open a file
`read()`	Yes (interface)	Yes	Read from a file



Key Takeaway

An API may or may not involve a system call. It depends on the function. Some APIs perform computations (no system call), while others request OS services (use system calls).



System Call Implementation

1 Each system call has a number

- A number is associated with each system call.
- The system-call interface maintains a table indexed according to these numbers.

2 Invoke the system call

- The system-call interface invokes the intended system call in the OS kernel and returns the status and any return values.

3 Caller need not know the details

- The programmer just needs to obey the API and understand what the OS will do as a result of the system call.

4 Details are hidden by the API

- Most details of the OS interface are hidden from the programmer by the API.
- Managed by the run-time support library (set of functions included with compiler).



User Application

```
e.g., fd = open("file.txt", ...);
```

1. API function call



Runtime Support Library (e.g., libc)

Converts API call to a system call (with system call number and parameters).

2. System call (number + parameters)

System-Call Interface

System Call Number	System Call
1	exit()
2	fork()
3	read()
4	write()
...	...
n	open()

Uses the system call number to find and invoke the correct system call in the OS kernel.

3. Invoke system call

4. Return status and any return values



Operating System Kernel

Performs the requested service (e.g., open()).



Key Takeaway

The API and run-time support library hide most of the details of system call implementation from the programmer.



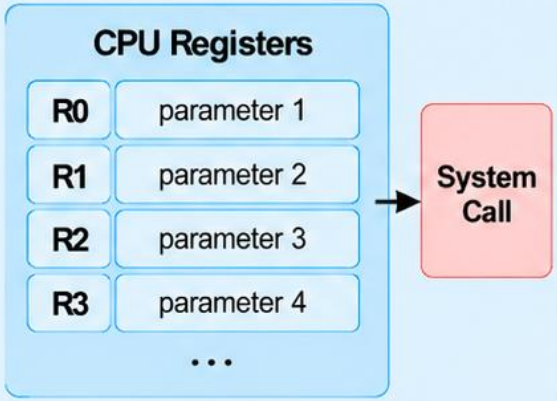


System Call Parameter Passing

A system call often requires more information than just the identity of the call.
The type and amount of information vary by operating system and call.

1 Pass Parameters in Registers

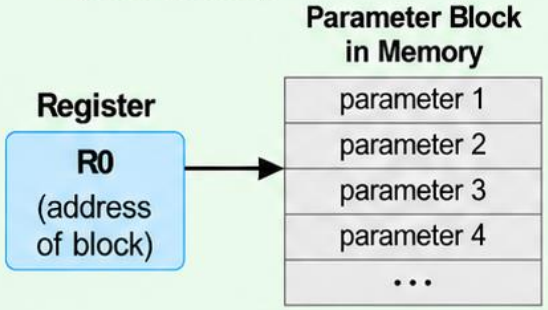
- Simplest approach.
- Parameters are placed directly in CPU registers.



Used when the number of parameters is small.

2 Pass a Block (Table) in Memory

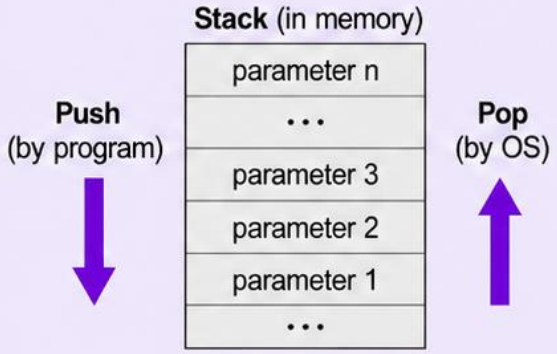
- Parameters are stored in a block (or table) in memory.
- The address of the block is passed as a parameter in a register.
- Used by Linux and Solaris (and others).



Useful when there are more parameters than registers.

3 Pass Parameters on a Stack

- Parameters are pushed onto a stack by the program and popped off by the operating system.
- Does not limit the number or length of parameters.



Used because it can handle any number or length of parameters.

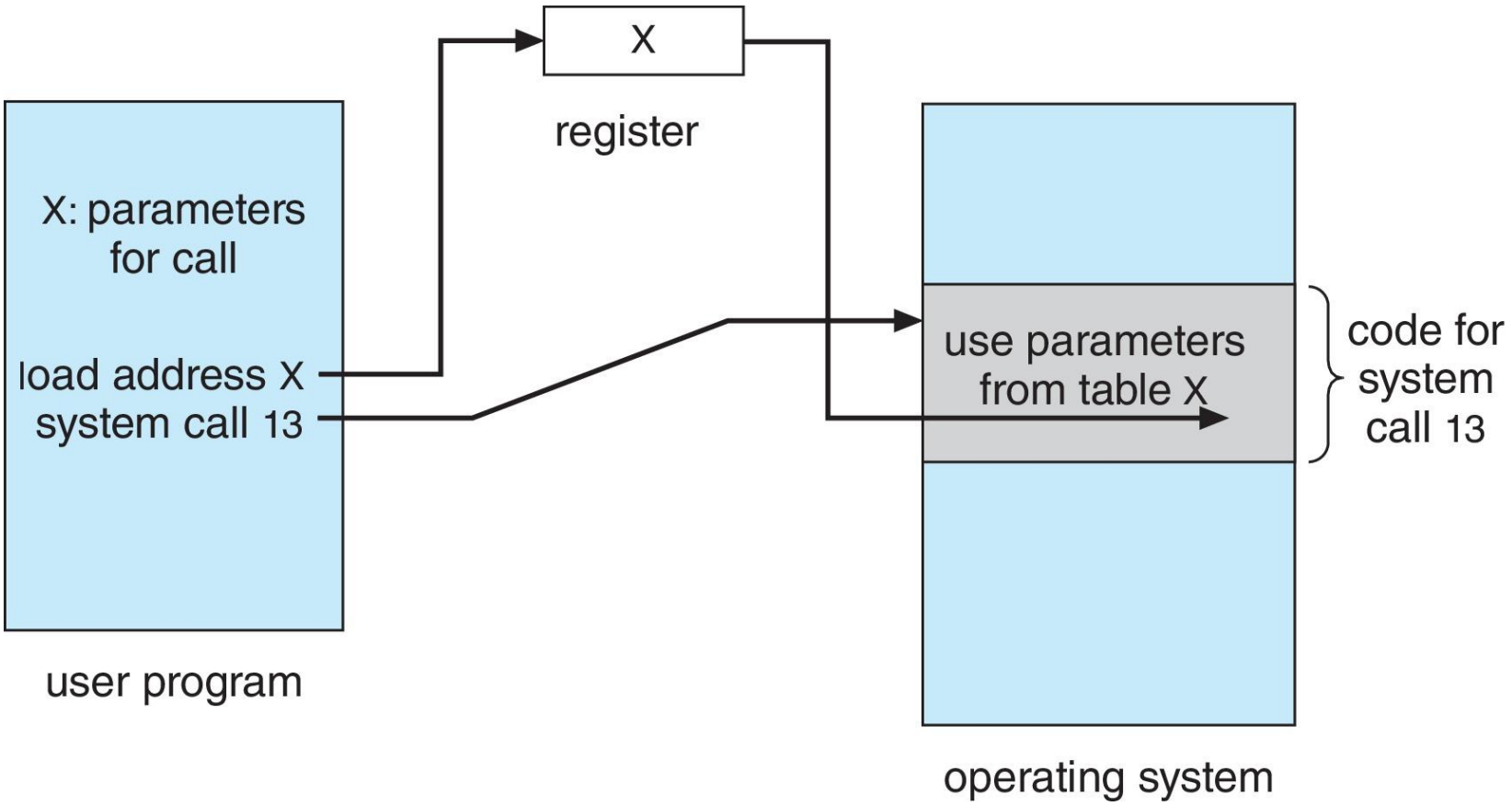
Key Takeaway

Three general methods are used to pass parameters to the operating system: **registers**, a **block in memory**, and a **stack**. The block and stack methods do not limit the number or length of parameters.





Parameter Passing via Table







Types of System Calls

System calls can be grouped into six major categories, each providing services for different operating-system functions.

1 Process Control

Manage the creation, execution, and termination of processes.



- Create process
- Terminate process
- Load and execute
- Get/set process attributes
- Wait for time or event
- Signal event
- Allocate and free memory
- Debug (e.g., single step)
- Locks for shared data

2 File Management

Create, delete, read, write, and manipulate files and directories.



- Create file
- Delete file
- Open and close file
- Read and write file
- Reposition file pointer
- Get and set file attributes
- Create and delete directory
- List directory contents

3 Device Management

Request and release I/O devices and perform I/O operations.



- Request device
- Release device
- Read from device
- Write to device
- Get and set device attributes
- Attach or detach device

4 Information Maintenance

Obtain and set information about the system, processes, and devices.



- Get system time and date
- Get and set system data
- Get and set process data
- Get and set device attributes
- Get and set file attributes
- Statistics and accounting information

5 Communications

Exchange information between processes, either on the same computer or over a network.



- Create communication link
- Send and receive messages
- Shared memory operations
- Socket operations (for network communication)

6 Protection

Control access to system resources and ensure security.



- Set and get permissions
- Allow and deny access
- Authenticate user (e.g., password)
- Create and delete protection domain
- Audit and log security events



Key Takeaway

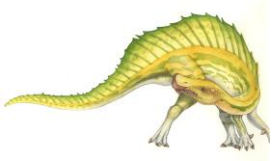
System calls are provided in six major categories: **process control**, **file management**, **device management**, **information maintenance**, **communications**, and **protection**.





Types of System Calls

- Process control
  - create process, terminate process
  - end, abort
  - load, execute
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - **Debugger** for determining **bugs, single step** execution
  - **Locks** for managing access to shared data between processes





Types of System Calls (Cont.)

- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices





Types of System Calls (Cont.)

- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices





Types of System Calls (Cont.)

- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access



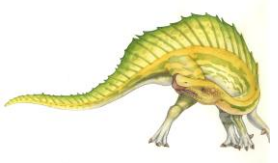


Examples of Windows and Unix System Calls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	`CreateProcess()`	`fork()`
	`ExitProcess()`	`exit()`
	`WaitForSingleObject()`	`wait()`
File management	`CreateFile()`	`open()`
	`ReadFile()`	`read()`
	`WriteFile()`	`write()`
	`CloseHandle()`	`close()`
Device management	`SetConsoleMode()`	`ioctl()`
	`ReadConsole()`	`read()`
	`WriteConsole()`	`write()`
Information maintenance	`GetCurrentProcessID()`	`getpid()`
	`SetTimer()`	`alarm()`
	`Sleep()`	`sleep()`
Communications	`CreatePipe()`	`pipe()`
	`CreateFileMapping()`	`shm_open()`
	`MapViewOfFile()`	`mmap()`
Protection	`SetFileSecurity()`	`chmod()`
	`InitializeSecurityDescriptor()`	`umask()`
	`SetSecurityDescriptorGroup()`	`chown()`



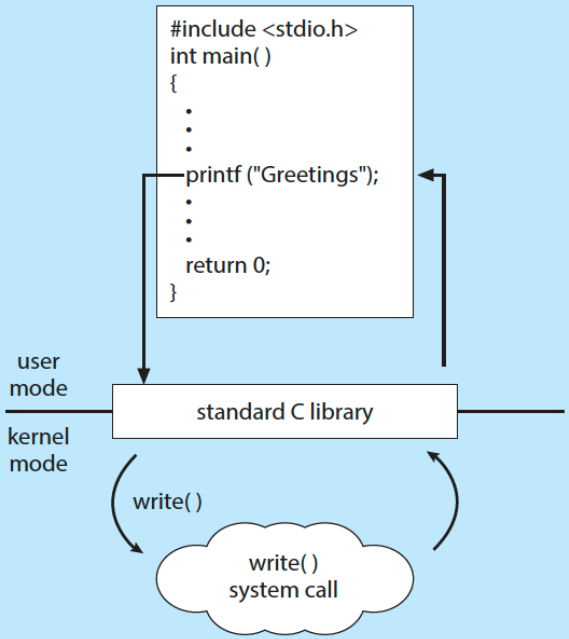


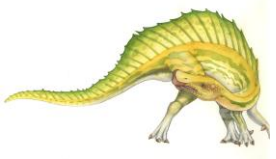
Standard C Library Example

- C program invoking printf() library call, which calls write() system call

THE STANDARD C LIBRARY

The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:



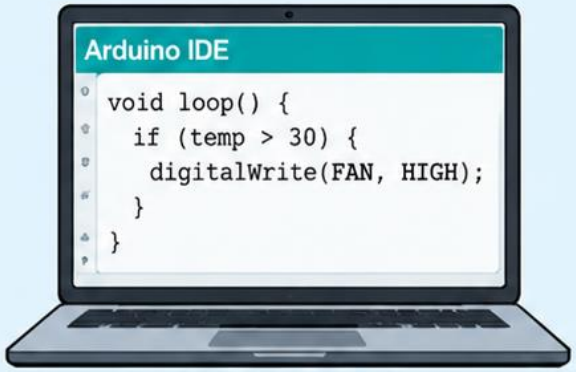




The Arduino Platform (No Operating System)

1 Write the program on a PC

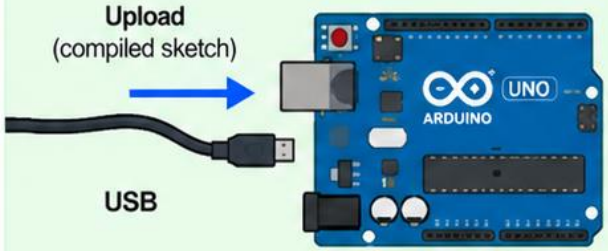
Write the Arduino program (sketch) using the **Arduino IDE** on a PC.



Program (sketch) on PC

2 Compile and upload via USB

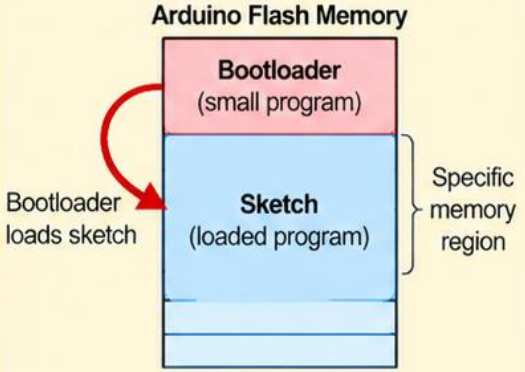
Compile the sketch and upload the compiled program to the Arduino's flash memory via a USB connection.



Arduino (microcontroller)

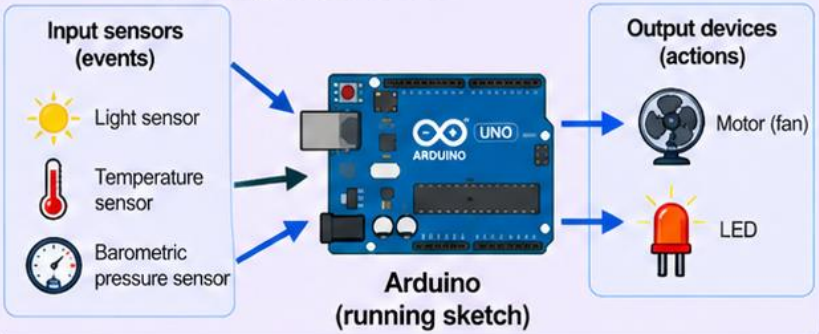
3 Bootloader loads the sketch

A small bootloader program loads the sketch into a specific region in the Arduino's flash memory.



4 Sketch runs and responds to events

Once loaded, the sketch begins running, waiting for events from input sensors and controlling output devices.



Arduino (running sketch)

5 Single-tasking system

- Only one sketch can be present in memory at a time.
- If another sketch is loaded, it replaces the existing sketch.
- No operating system.
- No user interface beyond hardware input sensors.

The Arduino is a simple, single-tasking system without an operating system.

6 Example behavior

If the temperature sensor detects that the temperature exceeds a threshold, the sketch may start the fan.



Start fan motor





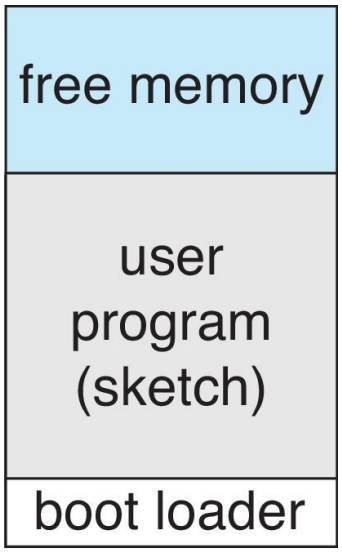
Example: Arduino

- Single-tasking
- No operating system
- Programs (sketch) loaded via USB into flash memory
- Single memory space
- Boot loader loads program
- Program exit -> shell reloaded



(a)

At system startup



(b)

running a program





Running a Program in FreeBSD

FreeBSD is multitasking: the shell can keep running while programs execute.

1 Login & Shell

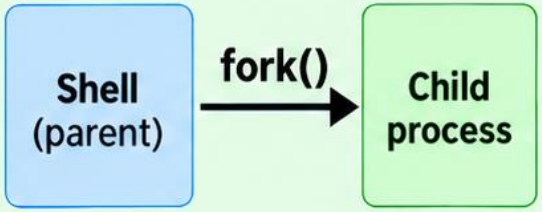
User logs in; shell waits for commands.

```
FreeBSD login: user
Password: ******

Welcome to FreeBSD!
$
```

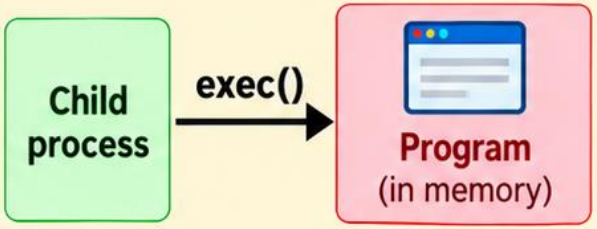
2 fork()

Shell creates a child process.



3 exec()

Child loads the selected program.



4 Run Program

Foreground: shell waits

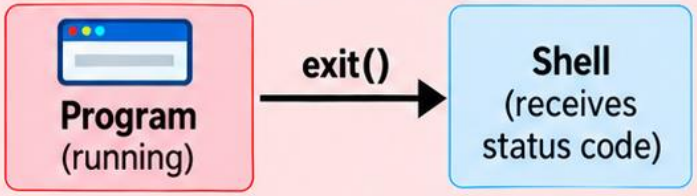
```
$ ls
file1  file2  file3
$
```

Background: shell continues

```
$ long_program &
[1] 12345
$
```

5 exit()

Program finishes and returns a status code.





Background jobs use files or GUI for I/O, not the keyboard.



Key Takeaway

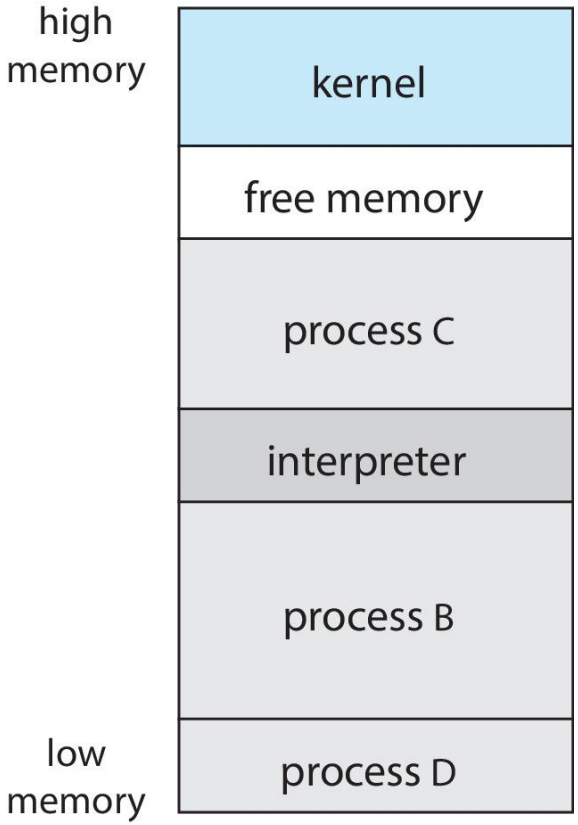
Shell uses **fork()** and **exec()**; FreeBSD lets multiple activities run at once.





Example: FreeBSD

- Unix variant
- Multitasking
- User login -> invoke user's choice of shell
- Shell executes fork() system call to create process
  - Executes exec() to load program into process
  - Shell waits for process to terminate or continues with user commands
- Process exits with:
  - code = 0 – no error
  - code > 0 – error code







System Services (System Utilities)

System services provide a convenient environment for program development and execution. They can be divided into several categories:

1 File Management

- Create, delete, copy, rename, print, list files and directories.



2 Status Information

- Get date, time, memory/ disk space, number of users.
- Provide performance, logging, debugging info.
- Use a registry for configuration information.



3 File Modification

- Text editors to create and modify files.
- Search or transform file contents.



4 Programming Language Support

- Compilers, assemblers, debuggers, interpreters (e.g., C, C++, Java, Python).



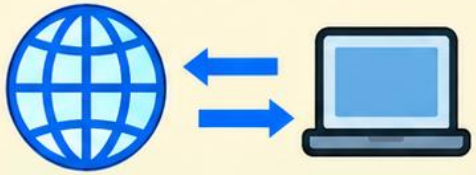
5 Program Loading and Execution

- Load compiled programs into memory.
- Provide loaders, linkage editors, and debugging tools.



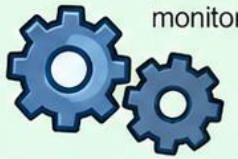
6 Communications

- Send messages, browse web pages, e-mail, remote login, transfer files.



7 Background Services

- Daemons/services started at boot time.
- Examples: network services, process schedulers, error monitoring, print servers.



Key Takeaway

System services (utilities) provide tools for file management, system information, file editing, program development and execution, communication, and background services.



System Services

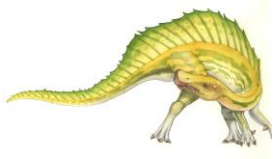
- System programs provide a convenient environment for program development and execution. They can be divided into:
  - File manipulation
  - Status information sometimes stored in a file
  - Programming language support
  - Program loading and execution
  - Communications
  - Background services
  - Application programs
- Most users' view of the operation system is defined by system programs, not the actual system calls





System Services (Cont.)

- Provide a convenient environment for program development and execution
  - Some of them are simply user interfaces to system calls; others are considerably more complex
- **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- **Status information**
  - Some ask the system for info - date, time, amount of available memory, disk space, number of users
  - Others provide detailed performance, logging, and debugging information
  - Typically, these programs format and print the output to the terminal or other output devices
  - Some systems implement a **registry** - used to store and retrieve configuration information





System Services (Cont.)

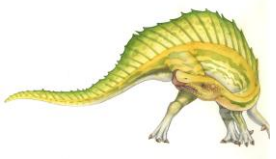
- **File modification**
  - Text editors to create and modify files
  - Special commands to search contents of files or perform transformations of the text
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
  - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another





System Services (Cont.)

- **Background Services**
  - Launch at boot time
    - Some for system startup, then terminate
    - Some from system boot to shutdown
  - Provide facilities like disk checking, process scheduling, error logging, printing
  - Run in user context not kernel context
  - Known as **services**, **subsystems**, **daemons**
- **Application programs**
  - Don't pertain to system
  - Run by users
  - Not typically considered part of OS
  - Launched by command line, mouse click, finger poke

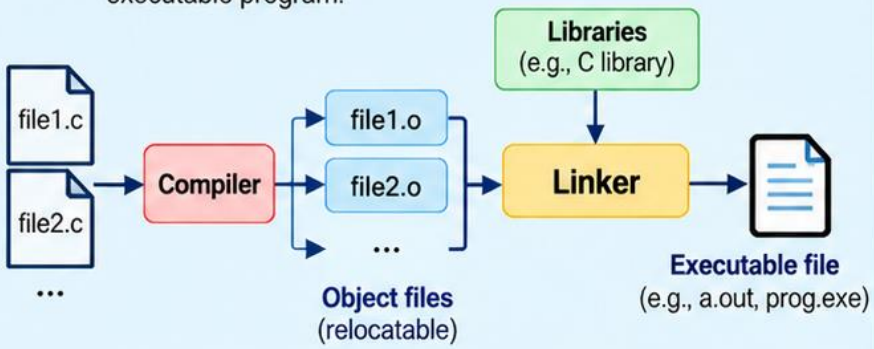




Linker and Loader

1 Linker

Combines one or more object files and libraries to create a final executable program.

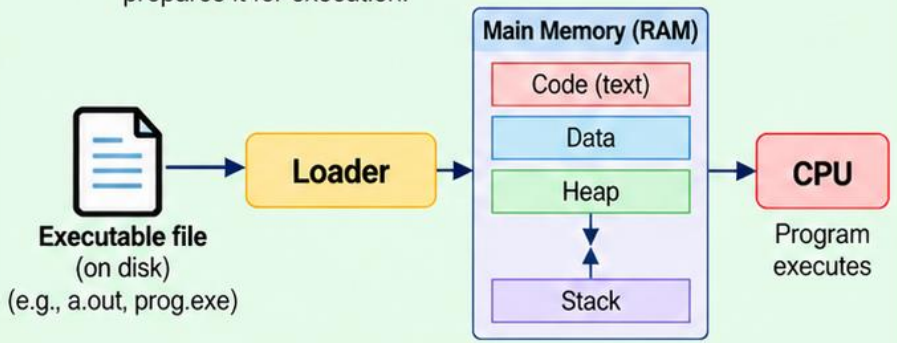


Main tasks of the linker:

- Combine object files
- Resolve external symbols (e.g., `printf`)
- Link required libraries
- Assign/adjust addresses (relocation information)
- Produce the final executable file

2 Loader

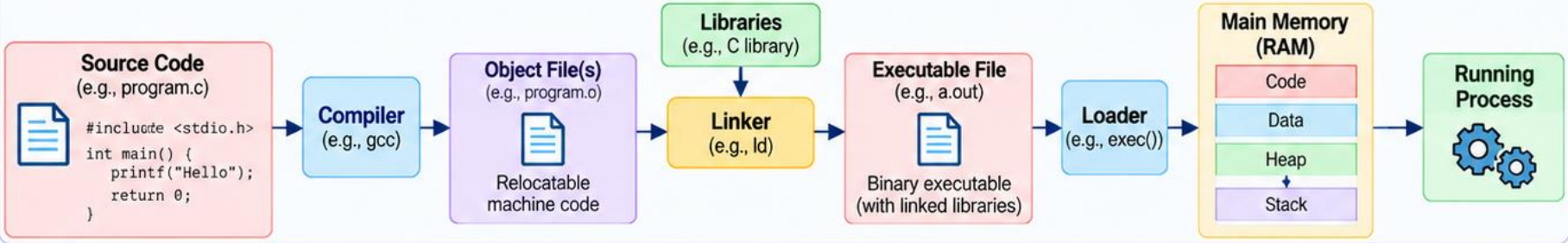
Loads the executable program into main memory (RAM) and prepares it for execution.



Main tasks of the loader:

- Load the executable into memory
- Set up the process's memory space (code, data, heap, stack)
- Perform required relocation
- Load required shared libraries (with help from the dynamic linker/runtime)
- Set the initial execution point
- Transfer control to the program

Complete Process: From Source Code to a Running Process



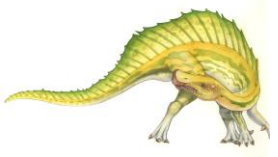
Key Takeaway

The **linker** creates the executable by combining object files and resolving references.
The **loader** loads the executable into memory and prepares it for execution.



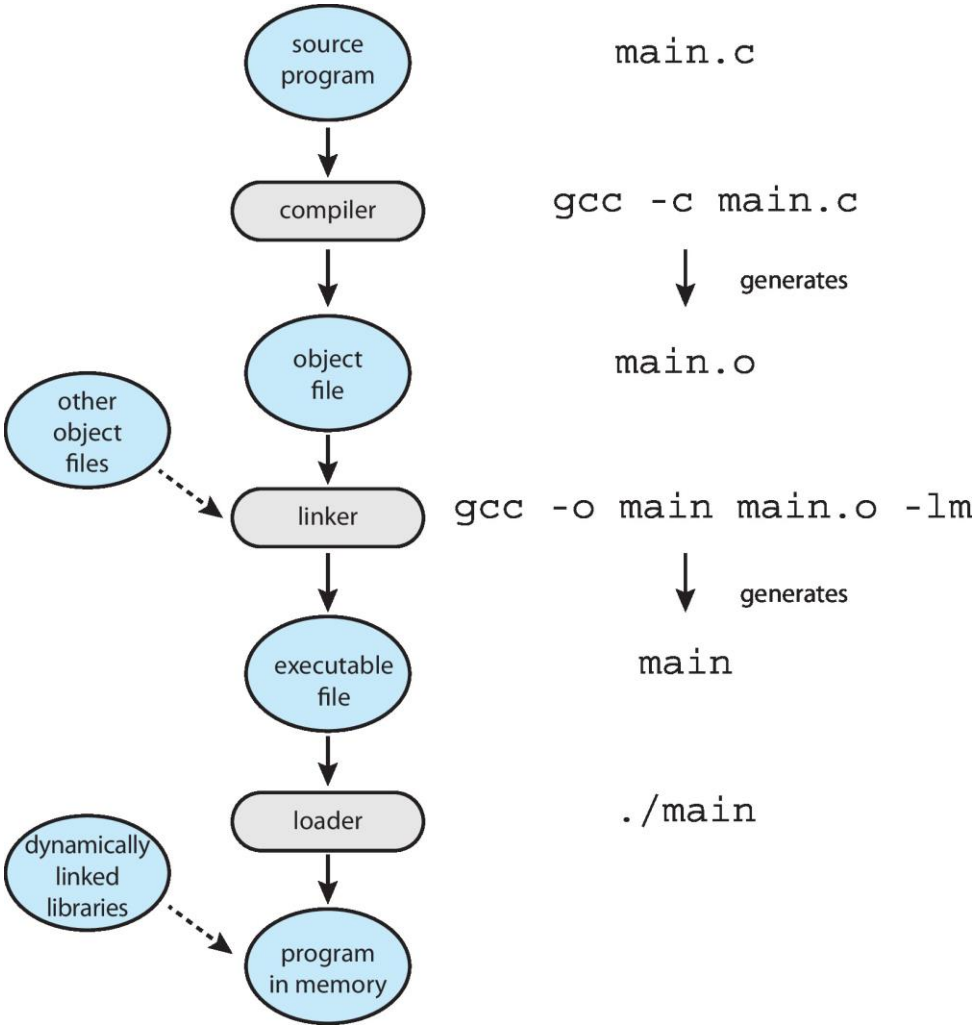
Linkers and Loaders

- Source code compiled into object files designed to be loaded into any physical memory location – **relocatable object file**
- **Linker** combines these into single binary **executable** file
  - Also brings in libraries
- Program resides on secondary storage as binary executable
- Must be brought into memory by **loader** to be executed
  - **Relocation** assigns final addresses to program parts and adjusts code and data in program to match those addresses
- Modern general purpose systems don't link libraries into executables
  - Rather, **dynamically linked libraries** (in Windows, **DLLs**) are loaded as needed, shared by all that use the same version of that same library (loaded once)
- Object, executable files have standard formats, so operating system knows how to load and start them





The Role of the Linker and Loader





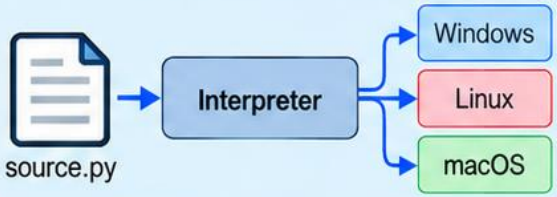


Why Applications Are Operating-System Specific

Applications compiled on one operating system usually cannot run on another because each OS has different system calls, binary formats, APIs, and often different CPU architectures.

1 Interpreted Languages (e.g., Python, Ruby)

- Same interpreter available on multiple OSs.
- Interpreter translates and executes code using the OS's native system calls.
- Lower performance and limited access to OS features.



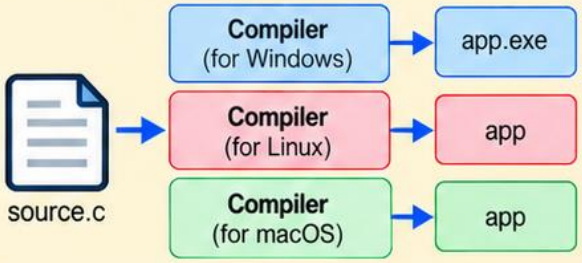
2 Virtual Machine / RTE (e.g., Java)

- Program runs inside a virtual machine (part of the language's runtime environment).
- Same RTE available on multiple OSs.
- Provides portability, but with some performance overhead and limited OS features.



3 Write for a Standard API (e.g., POSIX)

- Use a standard language/API.
- Compile separately for each OS.
- Porting can be time consuming and requires testing.



4 Other Barriers

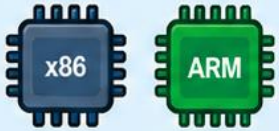
Binary format

Each OS uses a different executable file format (e.g., ELF, PE, Mach-O).



CPU instruction set

CPUs have different instruction sets (e.g., x86, ARM).



Different system calls

Each OS provides its own set of system calls with different names, parameters, and numbers.

open()	(Linux)
CreateFile()	(Windows)
open()	(macOS)

5 APIs and ABIs

API (Application level)

- Defines functions (e.g., GUI, file I/O).
- Applications using one OS's APIs may not work on another OS.

ABI (Architecture level)

- Defines low-level details (e.g., data sizes, calling conventions, binary format).
- Specific to an OS and CPU architecture, so it does not ensure cross-platform compatibility.

Key Takeaway

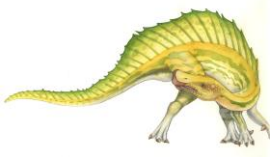
Differences in system calls, binary formats, CPU architectures, and APIs/ABIs make applications operating-system specific. Portability is possible using interpreters, virtual machines, or standard APIs, but it often comes with trade-offs.

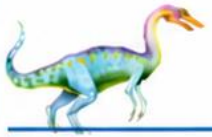




Why Applications are Operating System Specific

- Apps compiled on one system usually not executable on other operating systems
- Each operating system provides its own unique system calls
  - Own file formats, etc.
- Apps can be multi-operating system
  - Written in interpreted language like Python, Ruby, and interpreter available on multiple operating systems
  - App written in language that includes a VM containing the running app (like Java)
  - Use standard language (like C), compile separately on each operating system to run on each
- **Application Binary Interface** (**ABI**) is architecture equivalent of API, defines how different components of binary code can interface for a given operating system on a given architecture, CPU, etc.





2.7.1 Design Goals

Goal of OS Design

Define goals and specifications. The design is influenced by the choice of hardware and the type of system (desktop/laptop, mobile, distributed, or real time).

Key Point

There is no single set of requirements. Different environments have different needs, leading to different operating systems.

User Goals

Users want a system that is:

- Convenient to use
- Easy to learn and use
- Reliable
- Safe
- Fast

System (Developer) Goals

Developers want a system that is:

- Easy to design, implement, and maintain
- Flexible
- Reliable
- Error free
- Efficient

Challenges

- These requirements are often vague and open to interpretation.
- There is no unique solution for defining OS requirements.

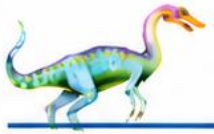
Examples

- **Wind River VxWorks:** real-time OS for embedded systems (different requirements).
- **Windows Server:** large multiaccess OS for enterprise applications (different requirements).

Conclusion

Specifying and designing an operating system is a highly creative task. Different requirements result in a wide variety of solutions for different environments.





2.7.2 Mechanisms and Policies

Basic Idea

Mechanism = **how**

Policy = **what**

Why Important?

- Separating them gives flexibility.
- Policies can change without changing the mechanism.

Example: Timer for CPU Protection

Timer (mechanism) — Sets CPU limit → **Timer value (policy)**: 10 ms, 50 ms, 100 ms, ...

Example: Program Priority

Scheduling mechanism (e.g., scheduler)

- **Policy 1**: Prioritize I/O-bound programs
- **Policy 2**: Prioritize CPU-bound programs

System Design Approaches

Microkernel-based OS

- Basic building blocks
- Flexible
- Policy-light

Windows / macOS (monolithic)

- More mechanism and policy built in

Linux (open source)

- Scheduler can be modified to support different policies

Key Takeaway

Policy decides **what**; mechanism decides **how**.
Separation improves flexibility.



2.7.3 Implementation

Overview

- An OS is a collection of many programs, written by many people over a long time.
- There is no single way to implement an OS.

Languages Used

- Early OSs: written in **assembly language**.
- Modern OSs: mostly in higher-level languages such as **C or C++**, with small parts in assembly.
- Often multiple languages are used.

Example: Android



Advantages of Higher-Level Languages

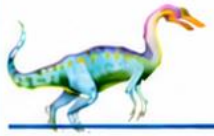
- Faster and easier development
- More compact code
- Easier to understand and debug
- Compiler improvements benefit the whole OS
- Easier to port to different hardware

Disadvantages

- May produce slower code
- May require more storage
  Usually not a major issue with modern compilers and processors.

Where Performance Matters

- Only a small part of the OS is critical to performance:
  - interrupt handlers
  - I/O manager
  - memory manager
  - CPU scheduler
- Performance improvements usually come from better data structures and algorithms, not just assembly code.



Modular Design of an Operating System

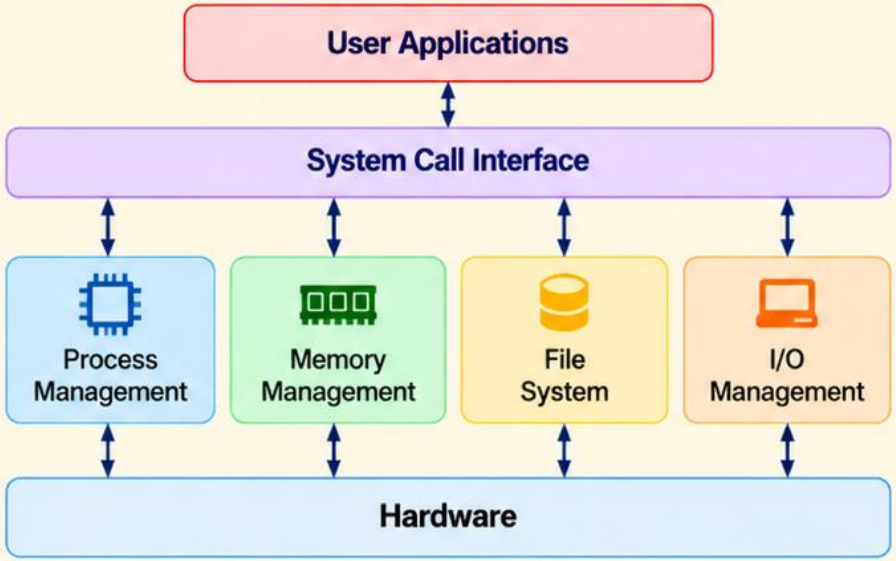
Key Idea

- A modern operating system is large and complex.
- It is easier to design, implement, and modify if the system is divided into small components (modules) rather than one single system.
- Each module should have a well-defined interface and function.

Benefits

- Easier to develop and understand
- Easier to maintain and modify
- Better organization
- Allows independent development of modules
- Changes in one module have limited impact on others

Modular Structure of an Operating System



Analogy: Your Own Program

Without Modularity

```
int main() {
   // all logic in one
   // big function
   ...
   return 0;
}
```

With Modularity

```
int add(int a, int b) {
   return a + b;
}

int main() {
   int result = add(3, 4);
   return 0;
}
```

- Instead of putting all code in `main()`, you separate logic into functions.
- Each function has clearly defined parameters and return values.
- `main()` calls these functions.

Key Takeaway

A large and complex operating system should be divided into small, well-defined modules with clear interfaces, making it easier to develop, maintain, and modify.



2.8.1 Monolithic Structure

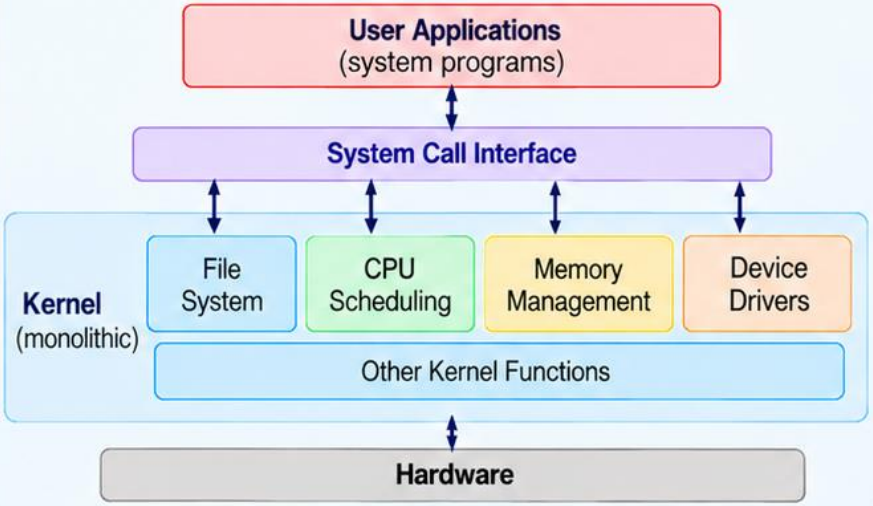
Key Idea

- All kernel functionality is combined into a single, static binary running in a single address space.
- This is called a **monolithic structure**.

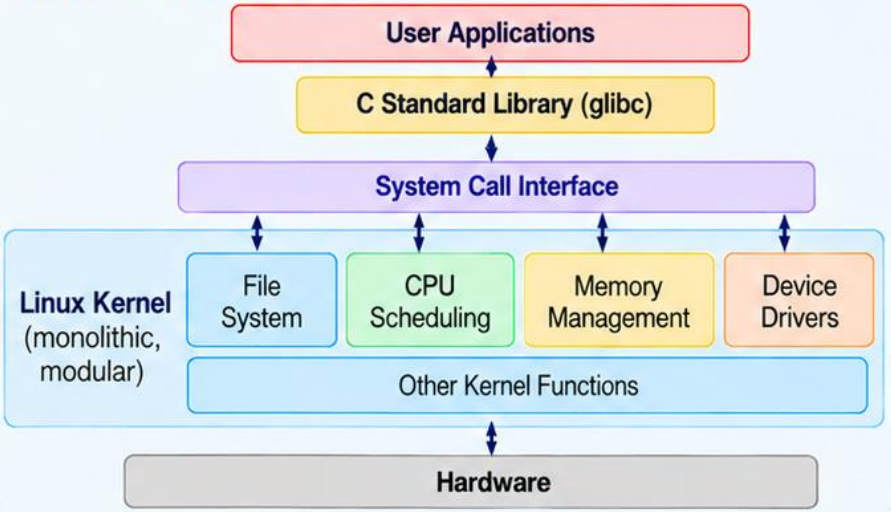
Characteristics

- Large kernel running in kernel mode.
- Provides services (file system, CPU scheduling, memory management, device drivers, etc.).
- Common in UNIX, Linux, and Windows.

Traditional UNIX Structure



Linux Structure



Advantages

- Very little overhead in the system-call interface.
- Fast communication within the kernel.
- Good performance and efficiency.

Disadvantages

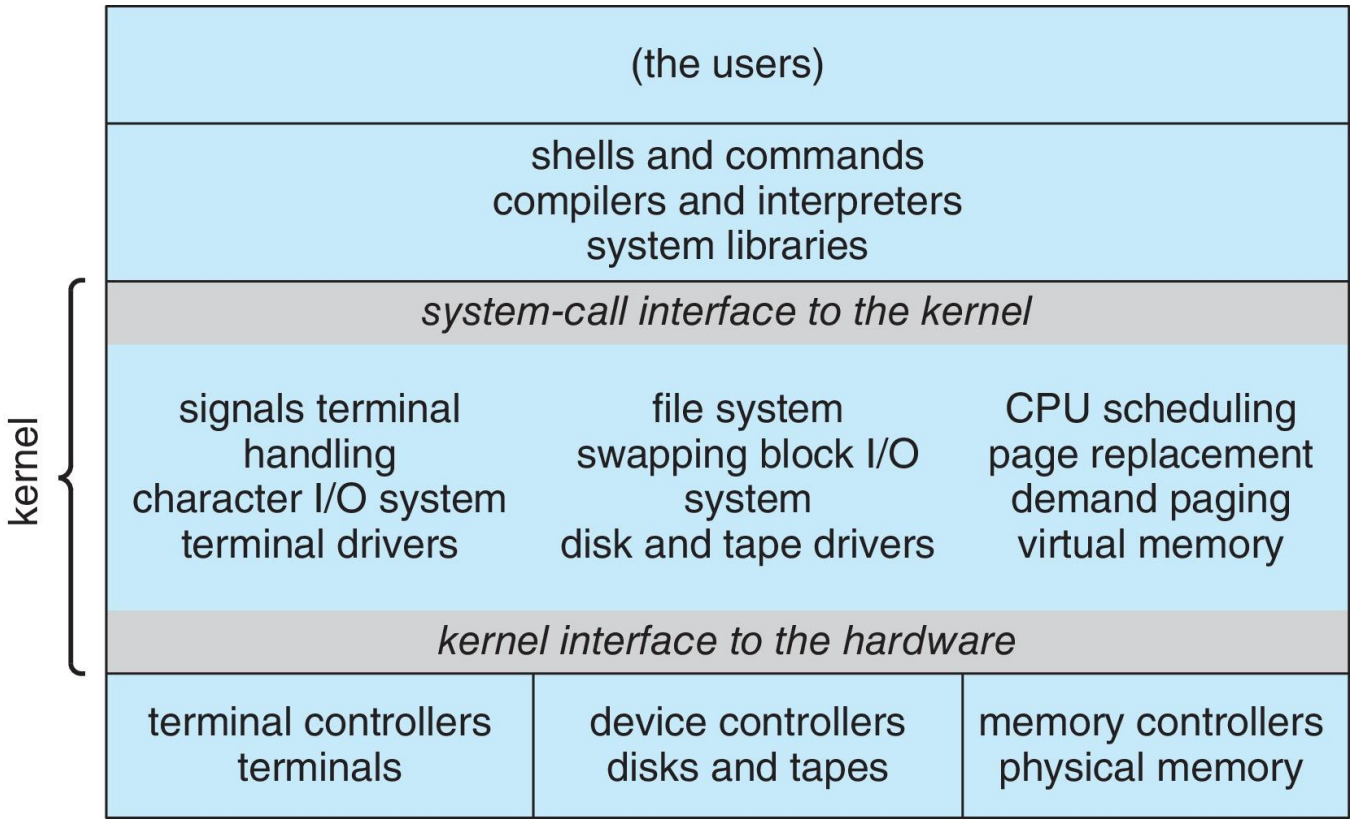
- Difficult to implement and extend.
- Large and complex code base.
- A bug in one part can affect the whole kernel (same address space).

Key Takeaway | Monolithic kernels are simple in structure and provide high performance, which is why they are still used in UNIX, Linux, and Windows.



Traditional UNIX System Structure

Beyond simple but not fully layered

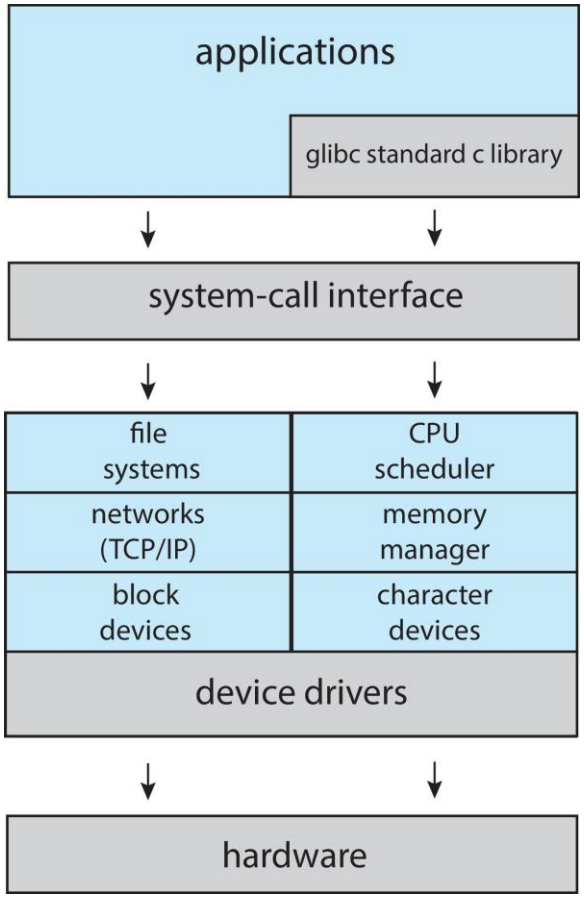


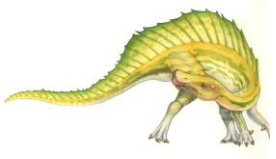




Linux System Structure

Monolithic plus modular design







2.8.2 Layered Approach

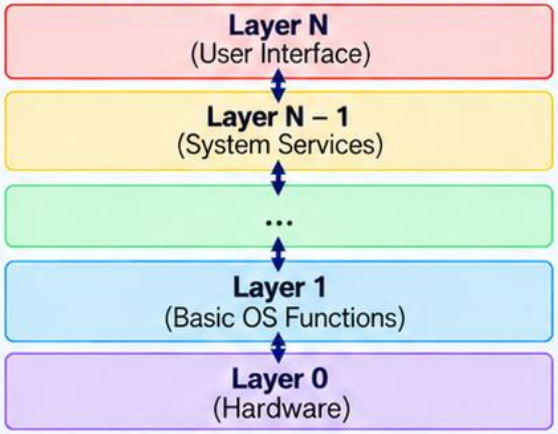
Key Idea

- Divide the OS into layers (levels).
- Each layer provides services to the layer above and uses services from the layer below.
- All layers together make up the kernel.

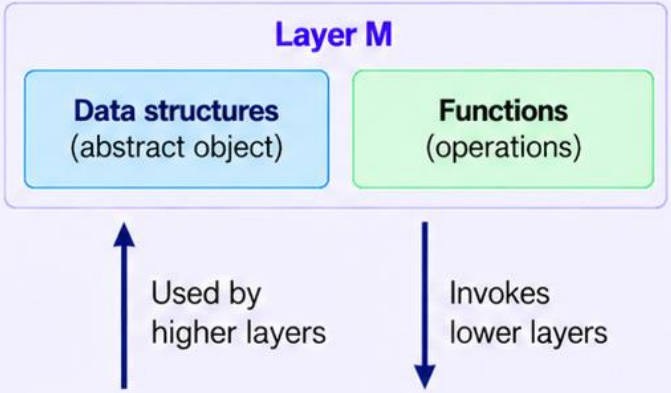
Why (Motivation)

- Loosely coupled: changes in one layer affect only that layer.
- Simpler construction, debugging, and verification.
- Each layer hides implementation details from higher layers.

Layered Structure of an Operating System



Layer M (Typical)



Advantages

- Simpler construction and debugging.
- Easier verification (debug each layer independently).
- Hides implementation details from higher layers.

Disadvantages

- Difficult to define the right functionality for each layer.
- More overhead (a request may pass through multiple layers).
- Pure layered approach is rarely used in practice.



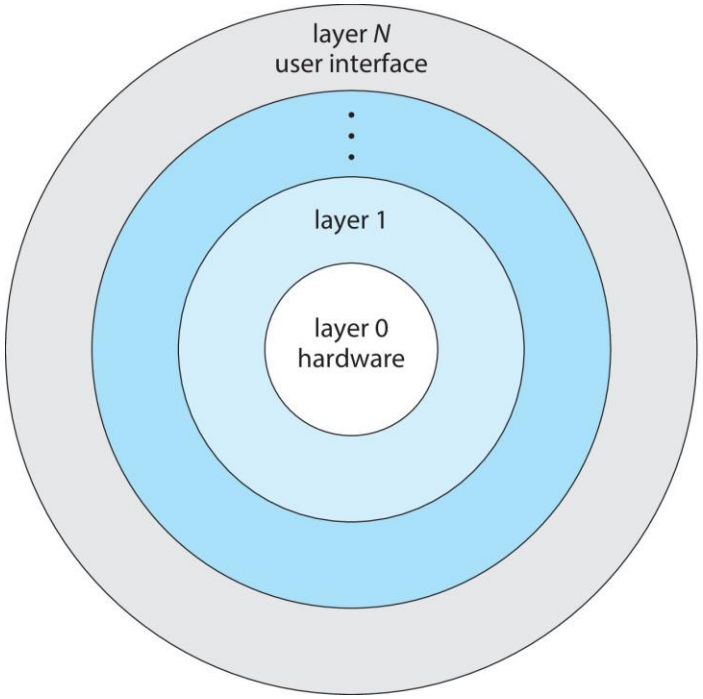
Key Takeaway

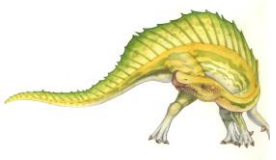
The layered approach provides a modular, loosely coupled design that is easier to build and debug, but can introduce performance overhead and is rarely used in pure form.



Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers

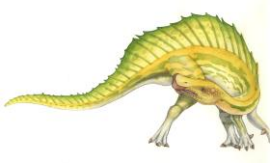


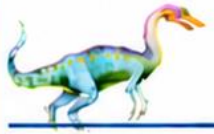




Microkernels

- Moves as much from the kernel into user space
- **Mach** is an example of **microkernel**
  - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- Detriments:
  - Performance overhead of user space to kernel space communication





2.8.3 Microkernels

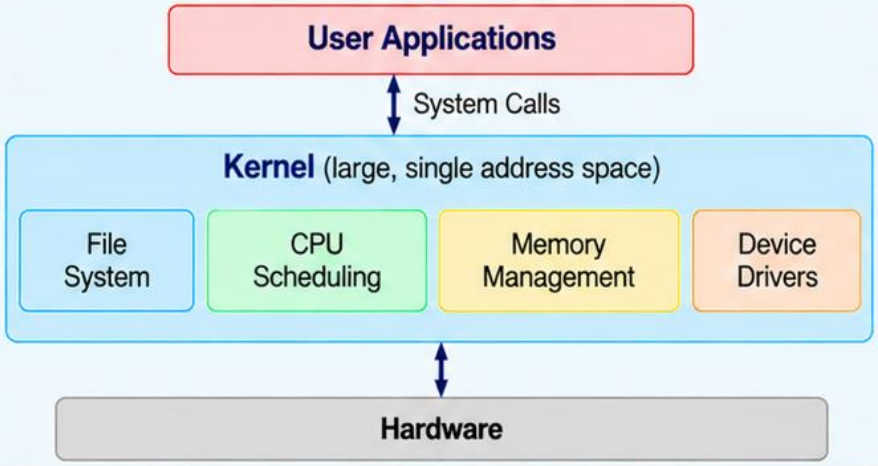
Key Idea

- Keep only essential components in the kernel.
- Move other services to user-level programs in separate address spaces.
- Results in a smaller kernel.

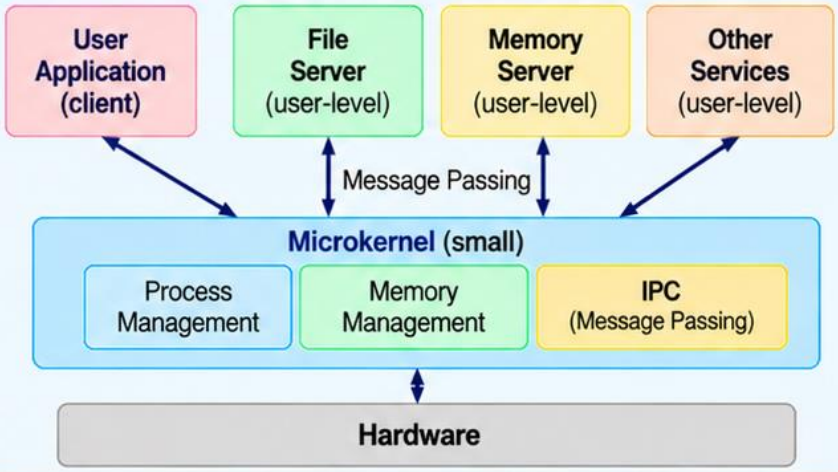
Typically in Microkernel

- Minimal process management
- Minimal memory management
- Inter-process communication (message passing)

Monolithic Kernel (e.g., traditional UNIX, Linux)



Microkernel (e.g., Mach, QNX, macOS Darwin)



Advantages

- Easier to extend (add new services in user space).
- More secure and reliable (services are user processes).
- Smaller kernel, easier to port to new hardware.

Disadvantages

- Higher overhead due to message passing.
- Copying messages between processes.
- Process switches are required.
- Can be slower than monolithic kernels.

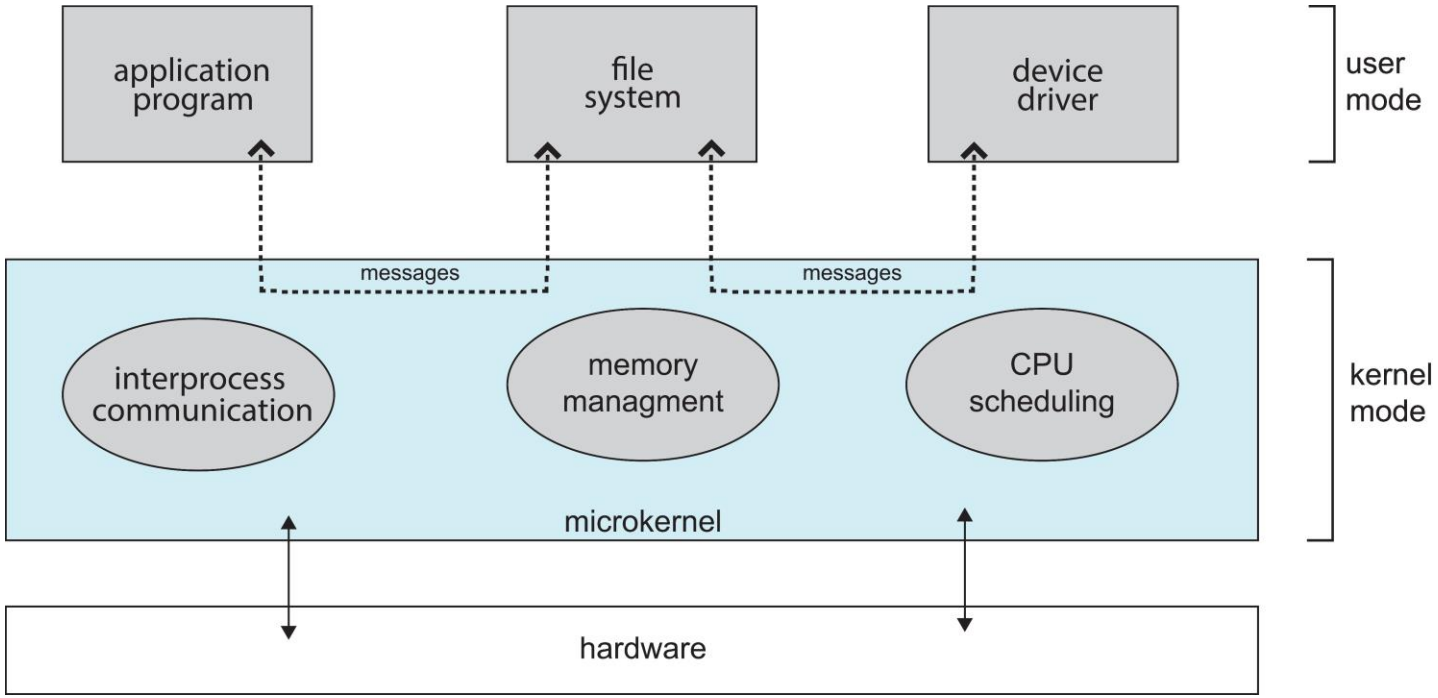


Key Takeaway

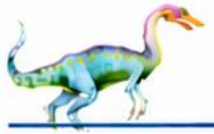
A microkernel keeps only essential services in the kernel and runs other services in user space, improving modularity and reliability but often with higher overhead.



Microkernel System Structure







2.8.4 Modules (Loadable Kernel Modules)

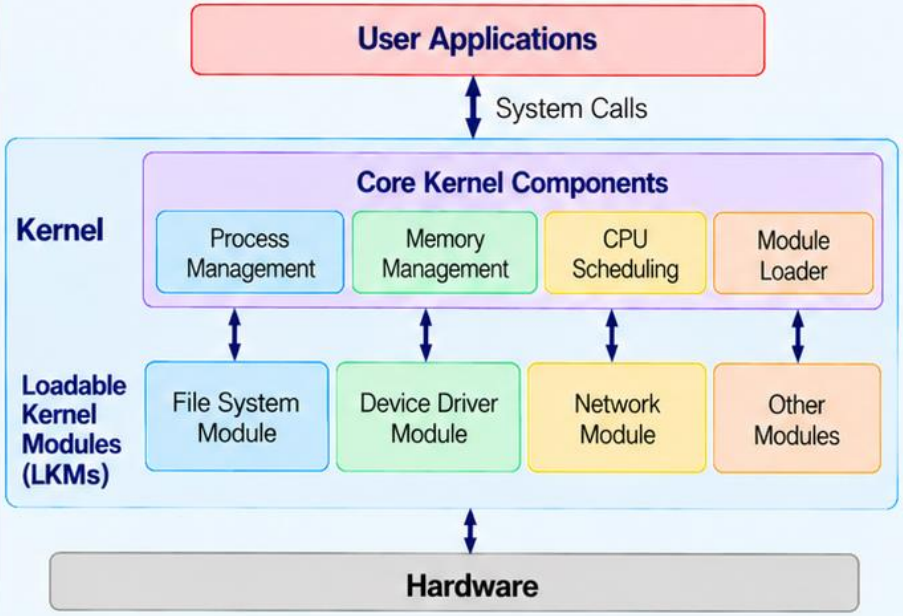
Key Idea

- Kernel has core components.
- Additional services can be linked in as **loadable kernel modules (LKMs)** at boot time or during runtime.
- Common in modern UNIX systems (Linux, macOS, Solaris) and Windows.

Why Use Modules?

- Add new features without recompiling the kernel.
- Flexible and extensible.
- Provides a modular design with protected interfaces (like layering).
- More efficient than microkernels (no message passing needed between modules).

Structure of a Modular Kernel



Example: Linux



- Uses LKMs mainly for device drivers and file systems.
- Modules can be inserted at boot time or during runtime (e.g., when a USB device is plugged in).
- Modules can also be removed during runtime.
- Provides a dynamic and modular kernel while maintaining the performance benefits of a monolithic system.

Advantages

- Flexible and extensible (add new services easily).
- No need to recompile kernel for every change.
- Maintains good performance (avoids message passing).
- Widely used in modern OSs (Linux, macOS, Solaris, Windows).

Disadvantages

- More complex than a simple monolithic kernel.
- Modules must be carefully designed and managed to maintain stability and security.



Key Takeaway | Loadable kernel modules allow a kernel to be extended with new services at runtime, combining the flexibility of modular design with the performance of a monolithic system.



2.8.5 Hybrid Systems

Key Idea

Hybrid systems combine **multiple OS** structures.
Goal: balance performance, security, and usability.

Linux (Hybrid)



- User Applications
- System Call Interface
- **Kernel** (monolithic + modules)
  - Core Kernel
  - Loadable Modules
- Hardware

- Monolithic kernel + modules
- Modules add functionality

Windows (Hybrid)



- User Applications
- Subsystems
  - Win32
  - POSIX
  - Other
- System Call Interface
- **Kernel** (largely monolithic)
  - Core Kernel
  - Loadable Modules
- Hardware

- Largely monolithic kernel
- Subsystems and modules

macOS, iOS, Android (Hybrid)


macOS

iOS


Android

- User Applications
- System Call Interface
- **Hybrid Kernel** (combines multiple ideas)
  - Core Kernel
  - Modules / Components
- Hardware

- Hybrid systems
- Combine multiple ideas



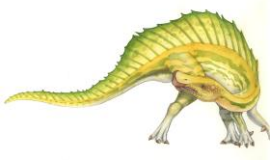
Key Takeaway

Real operating systems mix structures to get the best trade-offs.



Hybrid Systems

- Most modern operating systems are not one pure model
  - Hybrid combines multiple approaches to address performance, security, usability needs
  - Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality
  - Windows mostly monolithic, plus microkernel for different subsystem ***personalities***
- Apple Mac OS X hybrid, layered, **Aqua** UI plus **Cocoa** programming environment
  - Below is kernel consisting of Mach microkernel and BSD Unix parts, plus I/O kit and dynamically loadable modules (called **kernel extensions**)



2.8.5.1 macOS and iOS

macOS Architecture

(Desktop and Laptop Computers)



User Experience Layer
Aqua (GUI for mouse/trackpad)

Application Frameworks
Cocoa (macOS)

Core Frameworks
QuickTime, OpenGL, etc.

Kernel Environment (Darwin)
Mach microkernel + BSD kernel

Hardware
Intel (x86)

iOS Architecture

(iPhone and iPad)



User Experience Layer
Springboard (touch interface)

Application Frameworks
Cocoa Touch (iOS)

Core Frameworks
QuickTime, OpenGL, etc.

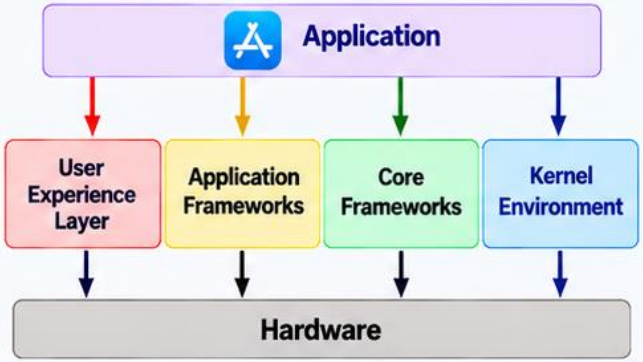
Kernel Environment (Darwin)
Mach microkernel + BSD kernel

Hardware
ARM (mobile devices)

Layers (Common to macOS and iOS)

1. **User Experience Layer**
   GUI for user interaction.
2. **Application Frameworks**
   Cocoa (macOS) or Cocoa Touch (iOS).
3. **Core Frameworks**
   Graphics, media (QuickTime, OpenGL, etc.).
4. **Kernel Environment (Darwin)**
   Mach microkernel + BSD kernel.
5. **Hardware**
   Intel (macOS) or ARM (iOS).

How Applications Use the System



Key Idea
Applications can use the user experience layer or bypass it and interact directly with lower layers.

Key Differences

macOS

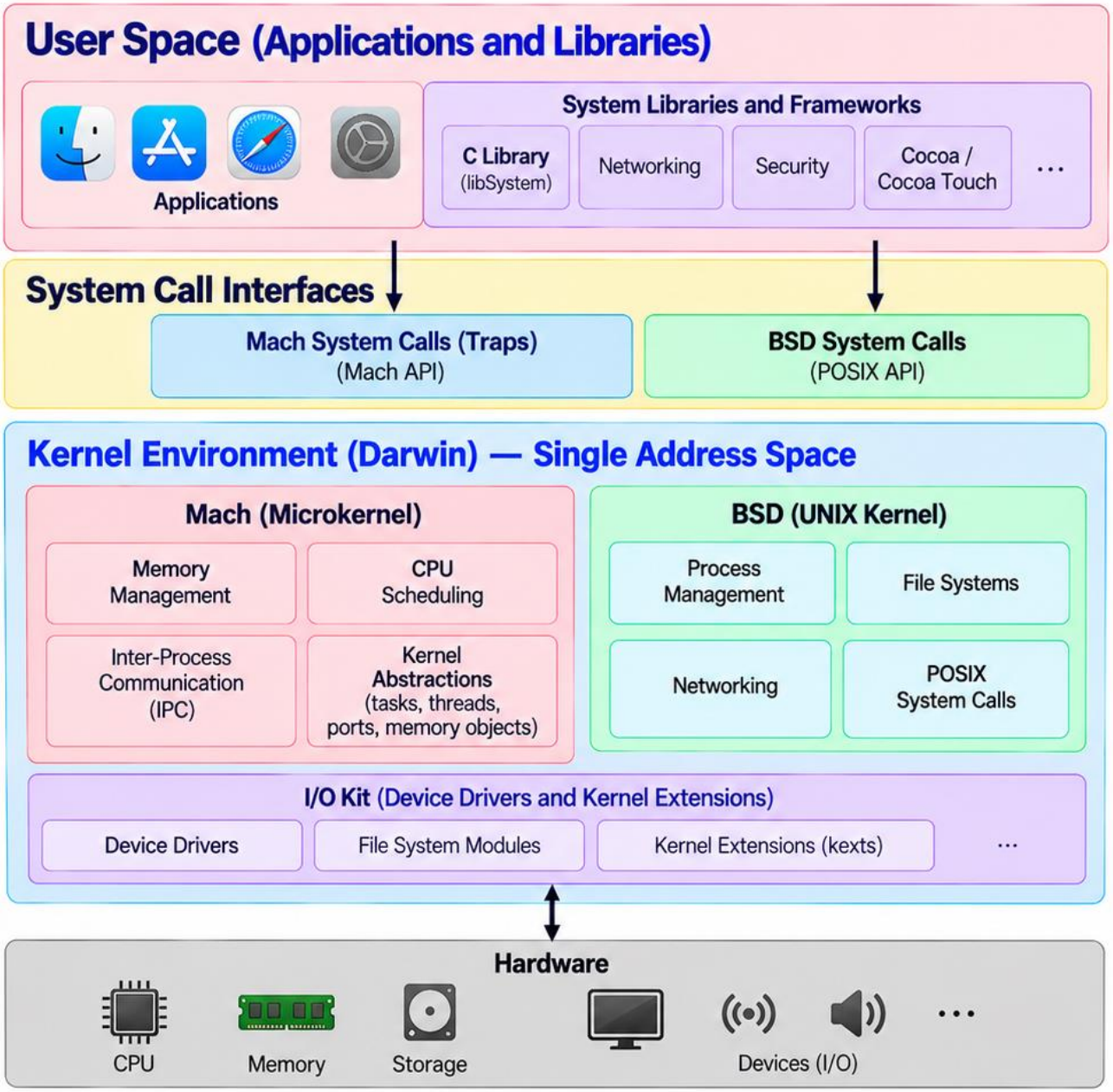
- For desktop and laptop computers
- Intel (x86) architecture
- Less restrictive for developers
- Full access to POSIX and BSD APIs
- General-purpose computing

iOS

- For mobile devices (iPhone, iPad)
- ARM architecture
- More restrictive for developers (may be closed)
- Limited access to POSIX and BSD APIs
- Additional features (e.g., power and memory management)
- Stricter security settings

Darwin Architecture (macOS / iOS)

A hybrid, layered system combining the Mach microkernel and the BSD UNIX kernel.



Key Idea

- Darwin uses a hybrid structure.
- It is a layered system consisting of Mach and the BSD UNIX kernel.
- Mach and BSD, along with the I/O kit and kernel extensions, run in a single kernel address space (not pure microkernel).

Two System Call Interfaces

- **Mach system calls (traps)**
  Provide Mach functionality.
- **BSD system calls**
  Provide POSIX functionality (e.g., fork()).

Mach (Microkernel Part)

- Memory management
- CPU scheduling
- Inter-process communication (IPC) (message passing, RPCs)
- Kernel abstractions: tasks, threads, ports, memory objects

BSD (UNIX Kernel Part)

- Process management
- File systems
- Networking
- POSIX system calls

I/O Kit and Kernel Extensions

- Provides device drivers and other modules.
- macOS uses kernel extensions (kexts) for loadable modules.

Android Architecture

Applications
Built-in apps and third-party apps (Java + Android API)

Phone | Messages | Browser | Camera | Other Apps

Application Framework
High-level services and APIs used by applications

Activity Manager | Window Manager | Content Providers | View System | ...

Android Runtime
Core set of libraries and Dalvik virtual machine

Core Libraries (e.g., collections, I/O, networking, etc.)

Dalvik Virtual Machine Executes Android bytecode (from Java)

Native Libraries
C/C++ libraries used by the framework and apps

WebKit (browser) | SQLite (database) | Media (multimedia) | Bionic libc (smaller libc) | ...

Hardware Abstraction Layer (HAL)
Provides a standard interface to hardware-specific modules

Camera HAL | Audio HAL | Sensor HAL | ...

Linux Kernel (modified)
Provides core OS services (with Android-specific modifications)

Process Management | Memory Management | Device-Driver Management | Power Management

Hardware
Mobile device hardware

CPU | Memory | Storage | Sensors | ...

Overview
- Developed by Open Handset Alliance (mostly Google)
- Open source

Kernel
- Based on Linux kernel (but modified)
- Provides process, memory, and device-driver management
- Adds power management and other mobile-specific features

Runtime Environment
- Includes core set of libraries and Dalvik virtual machine
- Apps developed in Java using Android API
- Java class files → Java bytecode → executable → runs in Dalvik VM

Native Libraries
- Include frameworks for web browser (WebKit), database (SQLite), multimedia, and a smaller libc

Similar to iOS
- Uses a similar layered stack as iOS
- Designed for mobile devices (smartphones and tablets)



Android

- Developed by Open Handset Alliance (mostly Google)
  - Open Source
- Similar stack to IOS
- Based on Linux kernel but modified
  - Provides process, memory, device-driver management
  - Adds power management
- Runtime environment includes core set of libraries and Dalvik virtual machine
  - Apps developed in Java plus Android API
    - Java class files compiled to Java bytecode then translated to executable than runs in Dalvik VM
- Libraries include frameworks for web browser (webkit), database (SQLite), multimedia, smaller libc

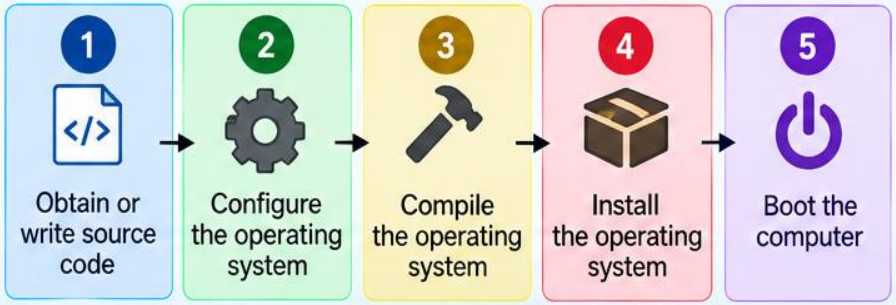


2.9 Building and Booting an Operating System

An operating system can be built specifically for one machine or for a class of machines with different hardware configurations.

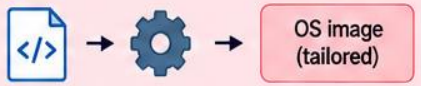
2.9.1 Operating-System Generation

To build an operating system from scratch, follow these steps:



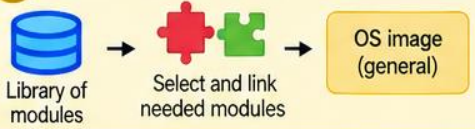
Ways to Generate an Operating System

1 Complete Compilation (System Build)



- Modify source code based on configuration.
- Compile entire system.
- Tailored to specific hardware.
- Smaller size, less general.

2 Precompiled Modules



- Select precompiled object modules from a library.
- Faster generation (no recompilation).
- More general, may not be fully tailored.

3 Fully Modular (Loadable Modules)



- Select modules at execution time.
- Very flexible (supports dynamic changes).
- Large size, most general.

Building Linux from Scratch

1. **Download source code**
   Get the Linux source code from http://www.kernel.org.
2. **Configure the kernel**
   Run `make menuconfig` to generate the `.config` file.
3. **Compile the kernel**
   Run `make` to compile the kernel. Produces `vmlinuz`.
4. **Compile kernel modules**
   Run `make modules` to compile the kernel modules.
5. **Install kernel modules**
   Run `make modules install` to install the modules.
6. **Install the new kernel**
   Run `make install` to install the kernel.

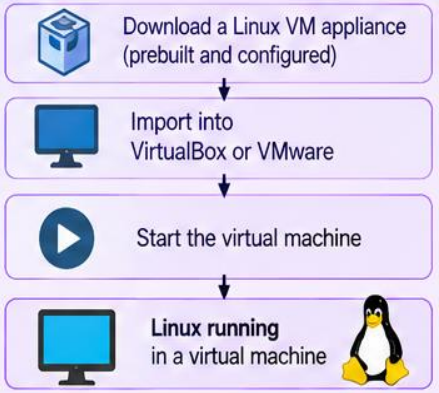
Installing Linux as a Virtual Machine

Instead of building from scratch, you can also run Linux in a virtual machine.

Option 1: Build a VM from Scratch



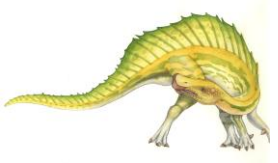
Option 2: Use a Prebuilt Appliance





Building and Booting an Operating System

- Operating systems generally designed to run on a class of systems with variety of perpherals
- Commonly, operating system already installed on purchased computer
  - But can build and install some other operating systems
  - If generating an operating system from scratch
    - Write the operating system source code
    - Configure the operating system for the system on which it will run
    - Compile the operating system
    - Install the operating system
    - Boot the computer and its new operating system





Building and Booting Linux

- Download Linux source code (http://www.kernel.org)
- Configure kernel via "`make menuconfig`"
- Compile the kernel using "`make`"
  - Produces `vmlinuz`, the kernel image
  - Compile kernel modules via "`make modules`"
  - Install kernel modules into `vmlinuz` via "`make modules_install`"
  - Install new kernel on the system via "`make install`"



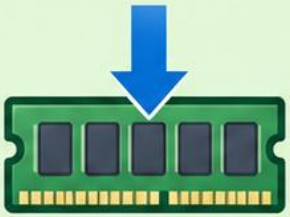
2.9.2 System Boot – Part 1

Booting loads the kernel and makes the operating system ready to use.

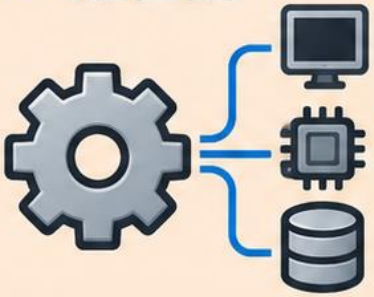
1. **Boot Loader Locates Kernel**



2. **Kernel Loads into Memory**



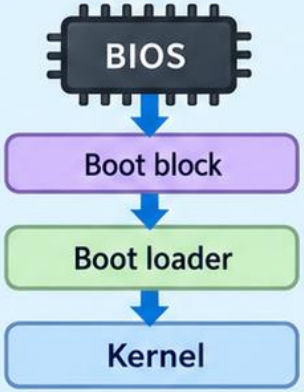
3. **Kernel Initializes Hardware**



4. **Root File System Mounted**

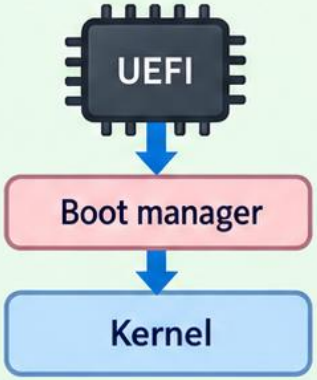


BIOS Boot



- multistage
- older systems

UEFI Boot



- single boot manager
- faster
- supports large disks

Boot loader finds the kernel; the kernel starts hardware and mounts the root file system.

Silberschatz, Galvin and Gagne ©2018

2.9.2 System Boot — Part 2

After the kernel loads, the OS starts services and shows the user interface.

Linux Boot Flow

1. GRUB reads configuration
2. Load compressed kernel + initramfs
3. Kernel initializes drivers
4. Switch to real root file system
5. Start systemd and services

Android Boot Flow

1. Vendor boot loader
2. Load Linux-based kernel + initramfs
3. Start init and Android services
4. Display home screen

Boot Loader Options

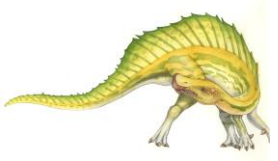
- choose kernel
- set boot parameters
- recovery / single-user mode

Linux commonly uses GRUB; Android uses vendor boot loaders.
Both load the kernel, start services, and reach the user interface.



System Boot

- When power initialized on system, execution starts at a fixed memory location
- Operating system must be made available to hardware so hardware can start it
  - Small piece of code – **bootstrap loader**, **BIOS**, stored in **ROM** or **EEPROM** locates the kernel, loads it into memory, and starts it
  - Sometimes two-step process where **boot block** at fixed location loaded by ROM code, which loads bootstrap loader from disk
  - Modern systems replace BIOS with **Unified Extensible Firmware Interface** (**UEFI**)
- Common bootstrap loader, **GRUB**, allows selection of kernel from multiple disks, versions, kernel options
- Kernel loads and system is then **running**
- Boot loaders frequently allow various boot states, such as single user mode





Operating-System Debugging

Debugging is the process of finding and fixing **errors (bugs)** and can also include performance tuning.

1 What is Debugging?

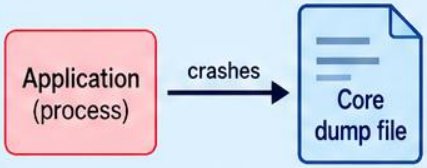
- Finding and fixing errors, or **bugs**
- Also includes **performance tuning** to optimize system performance
- Operating systems generate **log files** containing error information



2 Dump Files for Failures

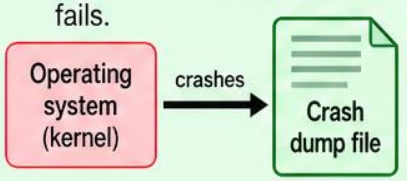
Core Dump (Application Failure)

- Captures the memory of the failed process.



Crash Dump (OS Failure)

- Contains kernel memory when the operating system fails.



3 Performance Tuning

Beyond crashes, performance tuning can help optimize system performance.

Trace Listings

- Record a listing of system activities for later analysis.
- Helps identify where time is spent and locate problems.

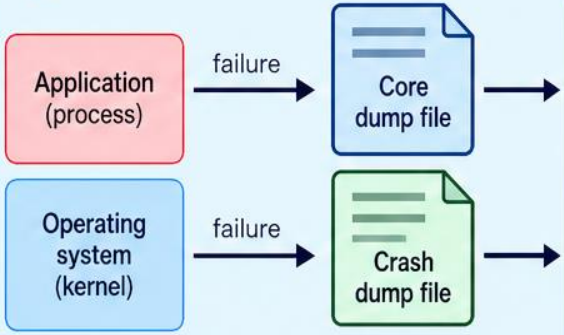


Profiling

- Periodic sampling of the instruction pointer.
- Used to find statistical trends and identify performance bottlenecks.



4 Overall View



Analysis and Debugging

- Examine log files and dump files
- Use trace listings to analyze activities
- Use profiling to find performance issues
- Fix errors and optimize performance



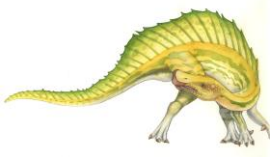
Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."



Operating-System Debugging

- **Debugging** is finding and fixing errors, or **bugs**
- Also **performance tuning**
- OS generate **log files** containing error information
- Failure of an application can generate **core dump** file capturing memory of the process
- Operating system failure can generate **crash dump** file containing kernel memory
- Beyond crashes, performance tuning can optimize system performance
  - Sometimes using ***trace listings*** of activities, recorded for analysis
  - **Profiling** is periodic sampling of instruction pointer to look for statistical trends

Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."



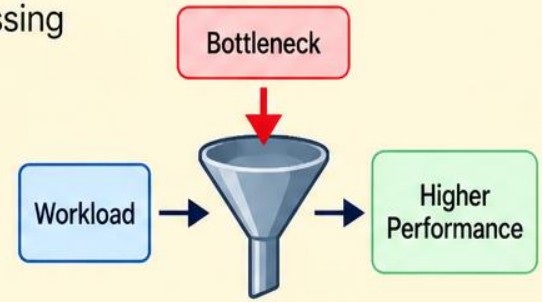


2.10.2 Performance Monitoring and Tuning

Performance tuning seeks to improve performance by removing processing bottlenecks.
To do this, we must monitor system performance to identify where the bottlenecks occur.

1 Goal: Improve Performance

- Find and remove processing bottlenecks.
- Requires monitoring system performance to identify bottlenecks.



2 What to Monitor?

Tools can provide:

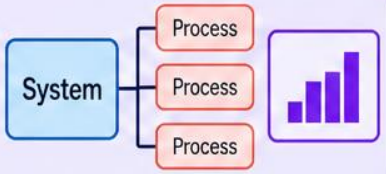
Per-process observations

- Performance information for a single process.



System-wide observations

- Overall system performance across all processes.

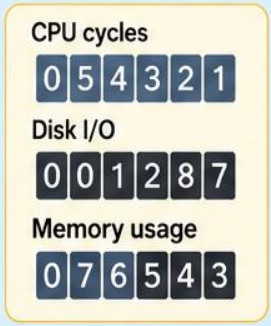


3 How to Monitor?

Tools use one of two approaches:

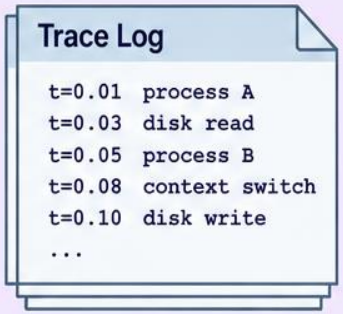
Counters

- Maintain counts of system events (e.g., CPU usage, I/O operations, memory usage).
- Provide quantitative measures of system behavior.



Tracing

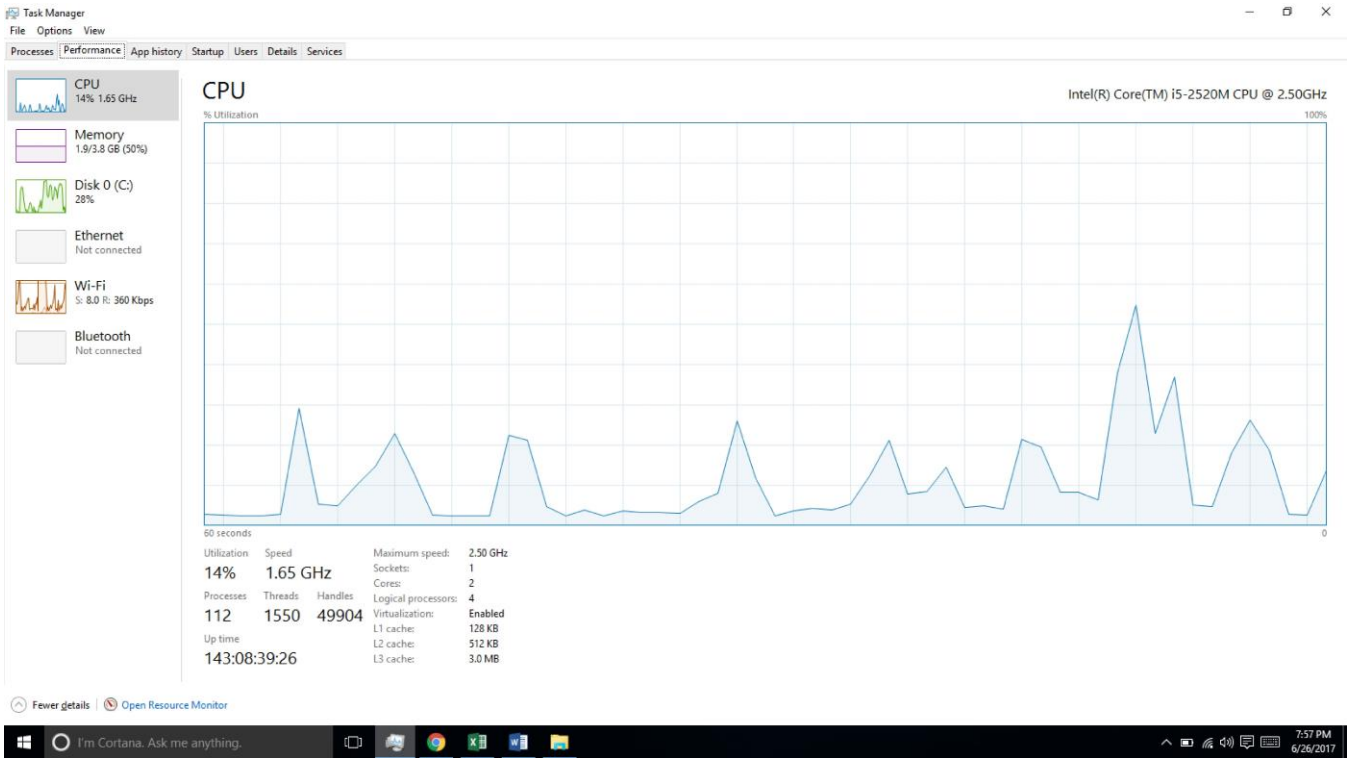
- Record a detailed listing of system activities (events) over time.
- Helps identify where time is spent and locate performance problems.





Performance Tuning

- Improve performance by removing bottlenecks
- OS must provide means of computing and displaying measures of system behavior
- For example, “top” program or Windows Task Manager







Tracing

- Collects data for a specific event, such as steps involved in a system call invocation
- Tools include
  - strace – trace system calls invoked by a process
  - gdb – source-level debugger
  - perf – collection of Linux performance tools
  - tcpdump – collects network packets



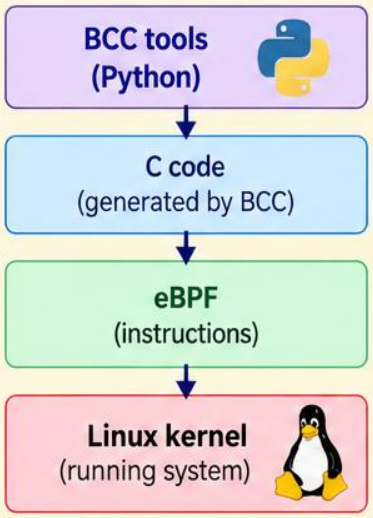


2.10.4 BCC

BCC (BPF Compiler Collection) is a toolkit for tracing and debugging Linux systems.
It uses eBPF to dynamically instrument a running system with low performance impact.

1 What is BCC?

- Rich toolkit for **tracing** and **performance monitoring** on Linux.
- Front-end interface to **eBPF** (extended Berkeley Packet Filter).
- Makes it easy to write tracing tools (using **Python**).
- **Low performance impact** and can be used on live production systems.



2 How It Works

1. Write tool in Python (uses C code template).
2. BCC compiles C code to eBPF instructions.
3. **Verifier** checks instructions for safety and correctness.
4. eBPF instructions are inserted into the running Linux kernel (using probes or tracepoints).

Key Points

- eBPF programs are written in a subset of C.
- Compiled into eBPF instructions.
- Verified before being inserted into the kernel (ensures safety and no negative impact).
- Can capture specific events (e.g., a system call) or monitor performance (e.g., disk I/O time).
- Uses **probes** or **tracepoints** to trace kernel events.

3 Example: disksnoop Tool

The **disksnoop** tool traces disk I/O activity.

```
$ ./disksnoop.py
TIME(s)          T      BYTES      LAT(ms)
--------------------------------------------
1946.29186700    R          8         0.27
1946.33965000    R          8         0.26
1948.34585000    W       8192         0.96
1950.43251000    R       4096         0.56
1951.74121000    R       4096         0.35
```

Output shows:

- Timestamp of the I/O operation.
- R or W (Read or Write).
- Number of bytes transferred.
- Latency (duration in ms) of the I/O.

4 Other BCC Tools

BCC provides many tools to trace different areas of the system.

opensnoop	Trace open() system calls
execsnoop	Trace process execution
fileslower	Trace slow file system operations
tcpconnect	Trace TCP connections
mysqld_qsl	Trace MySQL queries
javathreads	Trace Java thread activity
profile	CPU profiling

and many more ...

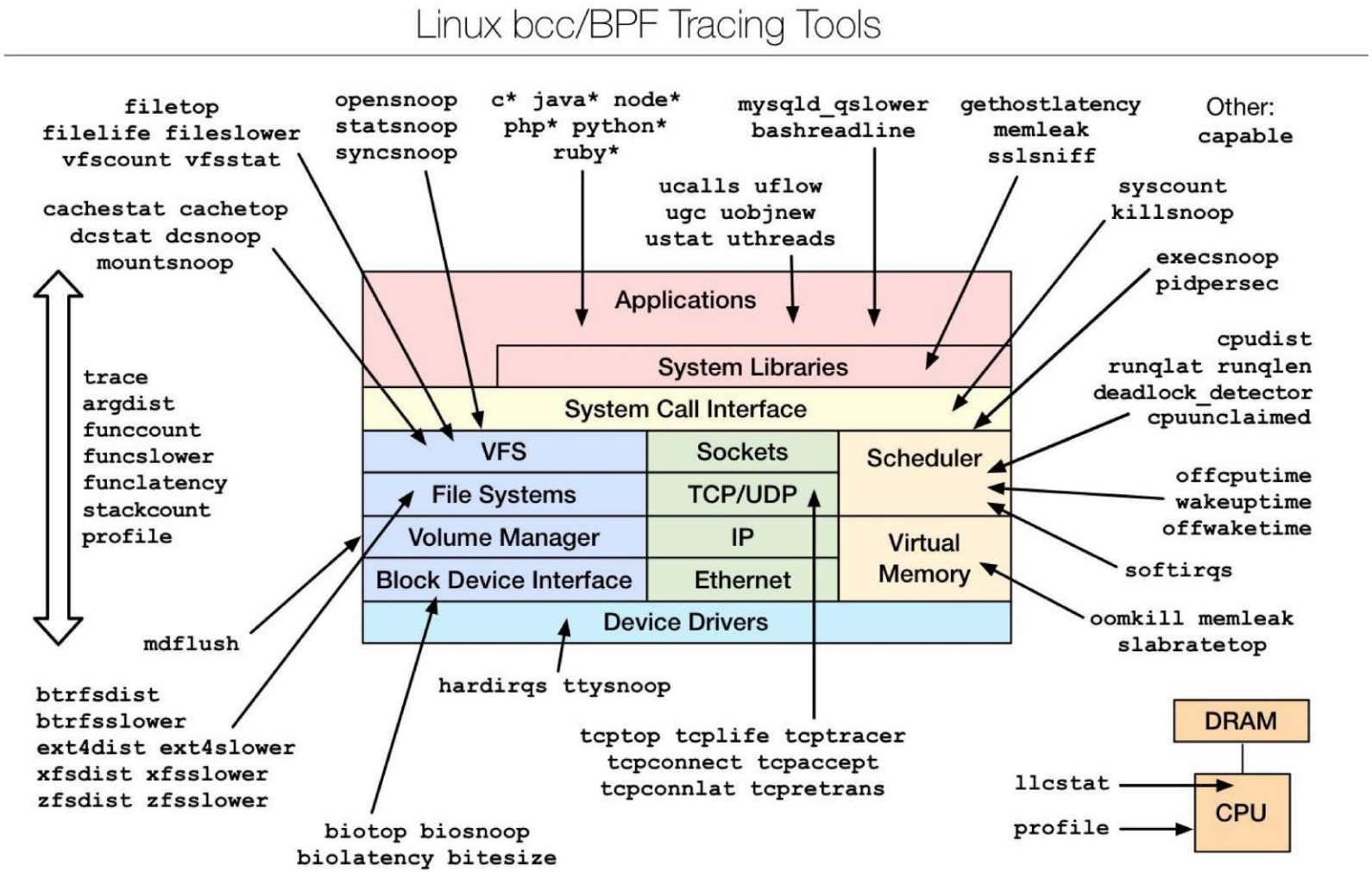
Example: Trace a specific process

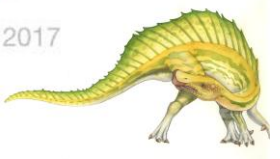
```
$ ./opensnoop -p 1225
```

Traces open() system calls performed only by the process with PID 1225.

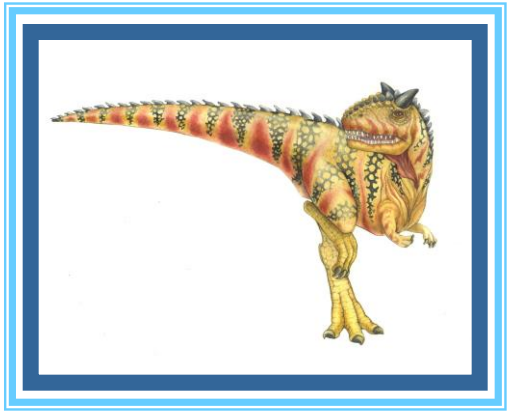


Linux bcc/BPF Tracing Tools





End of Chapter 2



Silberschatz, Galvin and Gagne ©2018